

C# Game Engine — Structured Lesson Book

34 Lessons · 2D & 3D Engine · 4 Complete Games · Unity Integration

Concept → Create files → Step-by-step code → Build & Run → Exercises → Quiz

■ Concept

■ Create File

■ Step-by-step

■ Build & Run

— Exercises

■ Quiz

Contents

PART 1 — C# FUNDAMENTALS

- Lesson 1** Variables, Types & Type Safety
- Lesson 2** Classes and Objects
- Lesson 3** Inheritance and virtual
- Lesson 4** Interfaces
- Lesson 5** Generics — List<T> and Dictionary<K,V>
- Lesson 6** Delegates and Events
- Lesson 7** Properties — get and set
- Lesson 8** Enums and Switch Expressions

PART 2 — 2D ENGINE SYSTEMS

- Lesson 9** The Game Loop and Delta Time
- Lesson 10** Sprite Rendering with SpriteBatch
- Lesson 11** Camera2D Transform Matrix
- Lesson 12** Input — Held vs Pressed vs Released
- Lesson 13** Scene Manager
- Lesson 14** AABB Collision Detection
- Lesson 15** AABB Collision Resolution
- Lesson 16** Physics Body — Velocity and Gravity
- Lesson 17** Audio — SFX and Music
- Lesson 18** Game State Machine

PART 3 — 3D ENGINE

- Lesson 19** Vector3 and 3D Space
- Lesson 20** The Three Matrices
- Lesson 21** Camera3D
- Lesson 22** Quaternions
- Lesson 23** Mesh and Vertex Buffers
- Lesson 24** Lighting — Phong Model
- Lesson 25** 3D Physics
- Lesson 26** Model Loading and Scene3D

PART 4 — GAMES

- Lesson 27** Platformer: Player Controller
- Lesson 28** Platformer: Tiles & Camera
- Lesson 29** Shooter: Movement & Aim
- Lesson 30** Shooter: Bullet Pool & Waves
- Lesson 31** FPS: Camera & Movement

Lesson 32 FPS: Raycasting & Interaction

Lesson 33 TPS: Orbit Camera & Movement

Lesson 34 TPS: Melee Combat & Enemy AI

PART 1

C# FUNDAMENTALS

Master the language before building the engine

LESSON

1

Variables, Types & Type Safety

How C# stores data — value types, reference types, and constants

■ Concept

Every piece of data in your engine — a position, a health value, a player name — is stored in a **variable**. C# is **statically typed**: every variable has a type that is fixed at compile time. The compiler uses this information to catch bugs before your game even runs.

There are two categories of types. **Value types** (int, float, bool, struct) store their data directly in the variable. When you copy a value type, you get a fully independent copy. **Reference types** (class, string, arrays) store a reference — essentially a memory address — to data on the heap. Two reference variables can point to the same object, so changing through one affects the other.

In engine code: positions and velocities are typically value types (float, or a Vector2 struct). Players, enemies, and scenes are reference types (classes). Understanding this difference prevents an entire class of subtle bugs where you expect two variables to be independent but they are not.

Type	Category	Copy independent?	Default
int	Value	Yes	0
float	Value	Yes	0.0f
bool	Value	Yes	false
string	Reference	No — shares data	null
class	Reference	No — shares data	null

■ Project Setup — Create your first C# console project



Terminal

```
dotnet new console -n Lesson01
cd Lesson01
dotnet run
```

■ CREATE FILE

Program.cs

The entry point. Replace all its content at each step.

1

Declare and print value-type variables

Program.cs

Open **Program.cs** and replace all its contents with the code below. Read every comment — these rules apply in every lesson from here on.

```
Program.cs C#
// Program.cs - Lesson 1, Step 1
int    score = 0;           // whole numbers, 32-bit integer
float  speed = 120.5f;      // 32-bit decimal. The 'f' suffix is REQUIRED
bool   alive = true;       // only true or false
double pi    = 3.14159265; // 64-bit decimal - more precise than float

Console.WriteLine($"score = {score}");
Console.WriteLine($"speed = {speed}");
Console.WriteLine($"alive = {alive}");
Console.WriteLine($"pi    = {pi}");
```

■ Build & Run

\$ dotnet run

Expected output:

```
score = 0
speed = 120.5
alive = True
pi    = 3.14159265
```

■ Tip

alive prints as True (capital T). C# booleans print True/False. The 'f' suffix disappears at runtime — it only tells the compiler which type to use.

■ Warning

Without the 'f' suffix, 120.5 is a 64-bit double. Assigning a double to a float causes a compile error — the compiler refuses the potential loss of precision.

2

Value copies vs reference copies

Program.cs

Add this block to the bottom of Program.cs — below the lines from Step 1. Do not replace Step 1.

```

Program.cs
// Step 2 – add below Step 1 code
// Value type copy: INDEPENDENT
int a = 10;
int b = a;      // b gets its own independent copy of 10
b = 99;         // changing b does NOT affect a
Console.WriteLine($"a={a}, b={b}"); // a=10, b=99

// Reference type copy: SHARED reference to the same array
int[] arr1 = { 1, 2, 3 };
int[] arr2 = arr1; // arr2 points to THE SAME array in memory
arr2[0] = 999;     // modifying through arr2 affects arr1
Console.WriteLine($"arr1[0]={arr1[0]}"); // 999 – same array!

// var: type is inferred at compile time (still statically typed)
var health = 100; // compiler infers: int
var name = "Hero"; // compiler infers: string
Console.WriteLine($"{name} has {health} HP");

```

■ Build & Run

\$ dotnet run

Expected output:

```

... (previous output) ...
a=10, b=99
arr1[0]=999
Hero has 100 HP

```

■ Tip

var does NOT mean dynamic typing. The type is locked at compile time from the right-hand side. In VS Code, hover over a var variable to see a tooltip showing the inferred type.

3

Constants and explicit casting

Program.cs

Add this block to the bottom of Program.cs. Constants appear throughout engine code for fixed values like GRAVITY, TILE_SIZE, and MAX_BULLETS.

```

Program.cs
C#
// Step 3 — add below Step 2 code
const float GRAVITY = 980f; // cannot change at runtime
const int TILE_SIZE = 32;
Console.WriteLine($"GRAVITY={GRAVITY}, TILE_SIZE={TILE_SIZE}");
// GRAVITY = 500f; // ERROR: cannot assign to a const

// (int) cast TRUNCATES toward zero — it does NOT round
float f1 = 3.9f; float f2 = -2.7f;
Console.WriteLine($"{(int)3.9f} = {(int)f1}"); // 3, not 4
Console.WriteLine($"{(int)-2.7f} = {(int)f2}"); // -2, not -3

// Use Math.Round() when you want rounding behaviour
Console.WriteLine($"Round(3.9f) = {(int)Math.Round(f1)}"); // 4
Console.WriteLine($"Round(-2.7f) = {(int)Math.Round(f2)}"); // -3

```

■ Build & Run

\$ dotnet run

Expected output:

```

... (previous output) ...
GRAVITY=980, TILE_SIZE=32
(int)3.9f = 3
(int)-2.7f = -2
Round(3.9f) = 4
Round(-2.7f) = -3

```

■ Warning

(int) truncates toward zero: 3.9 becomes 3 and -2.7 becomes -2. This matters when converting pixel positions or tile indices. Use Math.Round() when proper rounding is needed.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Declare 'float playerSpeed = 200f'. Print it. Change it to 150f. Print again.
- 2 Declare 'const int MAX_ENEMIES = 20'. Try adding a line that changes it — read the compiler error carefully.
- 3 Create int a=5 and int b=5. Set b=a. Change a to 99. Print both and explain why b is still 5.
- 4 Create int[] with 3 values. Assign it to arr2. Change arr2[0]=999. Print arr1[0] and explain the result.
- 5 Cast 7.8f and -1.3f with (int) and with Math.Round. Print all four results and explain each one.

■ Quiz

Q1. What does the 'f' suffix on 120.5f do?

A. Makes it negative

✓ Marks it as a 32-bit float literal

C. Makes it a constant

D. Causes rounding

Answer: B

Q2. When you write 'int b = a', what do you get?

A. Both share memory

✓ An independent copy — changing b does not affect a

C. a is cleared

D. A reference

Answer: B

Q3. What does const guarantee?

A. Heap allocation

B. Type inference

✓ Value cannot change at runtime

D. Thread safety

Answer: C

Q4. What does (int)3.9f produce?

A. 4

B. 3.0

✓ 3

D. Compile error

Answer: C

Q5. What does 'var health = 100' do?

A. Dynamic typing

✓ Infers int at compile time

C. Creates a nullable int

D. Prevents reassignment

Answer: B

LESSON

2

Classes and Objects

Blueprints for game entities — grouping data and behaviour together

■ Concept

A **class** is a blueprint that groups related data (**fields**) and behaviour (**methods**) in one place. An **object** is a specific instance created from that blueprint using **new**. Creating two Enemy objects gives you two independent enemies, each with their own internal state.

Encapsulation means hiding internal data with **private** so it can only be changed through controlled methods. This prevents bugs where other parts of your code set health to -99999 or speed to NaN. Private fields are conventionally prefixed with an underscore: `_health`, `_speed`.

Every engine system you build will be a class: Player, Enemy, Scene, Camera, AudioManager. Every tile, bullet, and pickup will be an object. Getting comfortable with classes now is the single most important thing you can do before writing engine code.

■ Project Setup — Add Enemy.cs to your Lesson01 project — no new project needed

■ CREATE FILE

Enemy.cs

New file — create in the same folder as Program.cs. C# compiles all .cs files automatically.

■ CREATE FILE

Program.cs

Updated — create and test Enemy objects.

1

Create the Enemy class

Enemy.cs

Create a new file called **Enemy.cs** in the Lesson01 folder. Add the code below. Read every comment — these patterns repeat in every engine class you will ever write.



Enemy.cs

C#

```
// Enemy.cs
public class Enemy
{
    // Private fields – underscore prefix is the C# convention
    // Only code inside this class can read or write these
    private int    _health;
    private float  _speed;
    private bool   _isDestroyed = false;

    // Constructor – same name as class, no return type
    // Called automatically when you write: new Enemy(50, 120f)
    public Enemy(int health, float speed)
    {
        _health = health;
        _speed  = speed;
    }

    // Public method – the ONLY way to reduce health from outside
    // This is encapsulation: control what can happen to private data
    public void TakeDamage(int amount)
    {
        _health -= amount;
        Console.WriteLine($"Enemy took {amount} dmg. HP: {_health}");
        if (_health <= 0)
        {
            _isDestroyed = true;
            Console.WriteLine("Enemy destroyed!");
        }
    }

    // Read-only properties – expose data, block direct writes
    // '=>' is shorthand for: get { return ...; }
    public bool  IsAlive      => _health > 0;
    public bool  IsDestroyed => _isDestroyed;
    public int   Health       => _health;
    public float Speed        => _speed;
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

■ Tip

dotnet build compiles without running. Use it to check for errors whenever you add a new file.

2

Create Enemy objects in Program.cs

Program.cs

Replace all content in **Program.cs** with the following. Two independent Enemy instances are created and we verify they are separate.

```
Program.cs
// Program.cs – Lesson 2, Step 2
// new Enemy() allocates on the heap and calls the constructor
Enemy goblin = new Enemy(health: 50, speed: 120f);
Enemy boss   = new Enemy(health: 500, speed: 40f);

Console.WriteLine($"Goblin alive: {goblin.IsAlive}"); // True
Console.WriteLine($"Boss alive:   {boss.IsAlive}");   // True

// Damage only the goblin
goblin.TakeDamage(30); // HP: 20
goblin.TakeDamage(30); // HP: -10 – this kills it

Console.WriteLine($"Goblin alive: {goblin.IsAlive}"); // False
Console.WriteLine($"Boss alive:   {boss.IsAlive}");   // True, untouched

// goblin._health; // COMPILER ERROR – _health is private
Console.WriteLine($"Boss HP: {boss.Health}"); // 500 – property OK
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
Goblin alive: True
Boss alive:   True
Enemy took 30 dmg. HP: 20
Enemy took 30 dmg. HP: -10
Enemy destroyed!
Goblin alive: False
Boss alive:   True
Boss HP: 500
```

3

Store enemies in a List and remove dead ones

Program.cs

In your engine you will always work with collections, not single variables. **Add this block to the bottom of Program.cs.**

```

Program.cs
// Step 3 — add below Step 2 code
Console.WriteLine("\n--- Wave of enemies ---");
var enemies = new List<Enemy>
{
    new Enemy(60, 80f), // will survive 50 damage (HP: 10)
    new Enemy(40, 120f), // will die to 50 damage (HP: -10)
    new Enemy(100, 50f), // will survive 50 damage (HP: 50)
};
Console.WriteLine($"Alive: {enemies.Count(e => e.IsAlive)}"); // 3

// Deal 50 damage to every enemy
foreach (var e in enemies)
    e.TakeDamage(50);

// RemoveAll is the safe way — never modify a list DURING foreach
enemies.RemoveAll(e => e.IsDestroyed);
Console.WriteLine($"Remaining: {enemies.Count}"); // 2

```

■ Build & Run

\$ dotnet run

Expected output:

```

--- Wave of enemies ---
Alive: 3
Enemy took 50 dmg. HP: 10
Enemy took 50 dmg. HP: -10
Enemy destroyed!
Enemy took 50 dmg. HP: 50
Remaining: 2

```

■ Warning

Never call `list.Remove()` or `list.RemoveAt()` inside a `foreach` loop. That throws `InvalidOperationException` at runtime because you are modifying a collection while it is being enumerated. `RemoveAll` runs after the loop — it is always safe.

➡ Exercises — Complete before moving on

➡ Exercises — Try These Now

- 1 Add string `Name` to `Enemy`. Set it in the constructor. Print the name inside `TakeDamage()`.
- 2 Add a `Heal(int amount)` method that increases `_health` but clamps it to the original starting value.
- 3 Create `Player.cs` with a `Player` class: `_health=100`, `_ammo=10`, and a `Shoot()` that reduces `ammo` by 1.
- 4 In `Program.cs`: create one `Player` and three `Enemies`. Call `Shoot()` once per enemy dealing 25 damage.
- 5 Print a final summary: enemies survived count and player's remaining ammo.

■ Quiz

Q1. What does 'new Enemy(50, 120f)' do?

A. Copies an enemy

✓ **Allocates on the heap and calls the constructor**

C. Value-type copy

D. Throws if exists

Answer: B

Q2. What does private do?

A. Read-only

B. Constant

✓ **Hides — only inside the class**

D. Prevents subclassing

Answer: C

Q3. What is a constructor?

A. Destroys objects

✓ **Called when new creates an object**

C. Static factory

D. Copy method

Answer: B

Q4. What does '=>' mean in 'public bool IsAlive => _health > 0'?

A. Assignment

B. Lambda

✓ **Expression body shorthand for get { return ...; }**

D. Null-conditional

Answer: C

Q5. Why not remove from list during foreach?

A. Too slow

✓ **Throws InvalidOperationException**

C. Silently skips

D. C# disallows

Answer: B

LESSON

3

Inheritance and virtual

Building specialised classes from a common base

■ Concept

In a game engine every drawable, updatable game object shares common behaviour: it has a position, can be active or destroyed, has an `Update()` method and a `Draw()` method. Instead of duplicating this in every class, you write a **base class** (`GameObject`) once, and all specific game objects **inherit** from it.

The **virtual** keyword on a base method signals 'children may replace this'. The **override** keyword in the child signals 'I am replacing it'. Storing mixed types (`Player`, `Enemy`, `Coin`) in a `List<GameObject>` and calling `Update()` on each is called **polymorphism** — C# dispatches the right version at runtime.

This is exactly how Unity's `MonoBehaviour` works. Unity's engine keeps a list of all `MonoBehaviour` instances and calls their `Update()` method each frame — the same pattern you are building now.

■ Project Setup — Add two files to Lesson01

■ CREATE FILE

GameObject.cs

The base class all engine objects inherit from.

■ CREATE FILE

Coin.cs

A specific game object that inherits from `GameObject`.

■ CREATE FILE

Program.cs

Updated to demonstrate polymorphism.

1

Create the `GameObject` base class

`GameObject.cs`

Create **`GameObject.cs`**. This class will be the parent of every game object in your engine.

```

GameObject.cs
// GameObject.cs
public class GameObject
{
    // Every game object has a position and a status
    public float X { get; set; }
    public float Y { get; set; }
    public bool IsActive { get; set; } = true;
    public bool IsDestroyed { get; set; } = false;

    // Constructor – subclasses can call this with base(x, y)
    public GameObject(float x = 0, float y = 0)
    { X = x; Y = y; }

    // virtual: children MAY override these
    // Default behaviour: do nothing
    public virtual void Update(float dt) { }
    public virtual void Draw() { }

    public override string ToString()
        => $"{GetType().Name} at ({X},{Y}) alive={IsActive}";
}

```

■ Build & Run

\$ dotnet build

Expected output:

```

Build succeeded.
    0 Warning(s)
    0 Error(s)

```

2

Create a Coin class that overrides Update and Draw

Coin.cs

Create **Coin.cs**. The Coin overrides Update to bob up and down using a sine wave.


```

Coin.cs
// Coin.cs
public class Coin : GameObject // ': GameObject' = inherit from it
{
    private float _baseY; // the resting Y position
    private float _bobTimer = 0f;
    private int _value;

    public Coin(float x, float y, int value) : base(x, y)
    // ': base(x, y)' calls the parent constructor with those args
    {
        _baseY = y;
        _value = value;
    }

    // override: REPLACES the parent's virtual Update
    public override void Update(float dt)
    {
        _bobTimer += dt;
        // Sine wave bob - moves ±4 pixels from base position
        Y = _baseY + Mathf.Sin(_bobTimer * 3f) * 4f;
    }

    public override void Draw()
        => Console.WriteLine($"Coin({_value}pt) at Y={Y:F1}");

    public void Collect()
    { Console.WriteLine($"Collected {_value} points!"); IsDestroyed = true; }
}

```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.

0 Warning(s)

0 Error(s)

■ Tip

' : base(x, y)' calls the parent constructor before the child's body runs. If you don't include it, the parent's default constructor is called instead.

3

Demonstrate polymorphism in Program.cs

Program.cs

Replace all content in **Program.cs** with the code below. A single list holds mixed types and calls the right method on each.

```
Program.cs
// Program.cs - Lesson 3
var scene = new List<GameObject>
{
    new Coin(100, 200, 10),
    new Coin(150, 180, 25),
    new Enemy(50, 80f),    // Enemy inherits GameObject too
};

Console.WriteLine("--- Simulating 3 frames ---");
for (int frame = 0; frame < 3; frame++)
{
    Console.WriteLine($"Frame {frame}:");
    foreach (var obj in scene)
        if (obj.IsActive)
        {
            obj.Update(0.016f); // ~60 FPS delta time
            obj.Draw();          // calls COIN's Draw or ENEMY's Draw
        }
}

// Collect the first coin
((Coin)scene[0]).Collect();
scene.RemoveAll(o => o.IsDestroyed);
Console.WriteLine($"Objects remaining: {scene.Count}"); // 2
```

■ Build & Run

\$ dotnet run

Expected output:

```
--- Simulating 3 frames ---
Frame 0:
Coin(10pt) at Y=200.0
Coin(25pt) at Y=180.0
Frame 1:
Coin(10pt) at Y=200.2
...
Collected 10 points!
Objects remaining: 2
```

■ Tip

obj.Draw() calls Coin.Draw() for coins and Enemy.Draw() (which prints nothing) for enemies. C# picks the right version at runtime — this is polymorphism.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Create a Player class that inherits from GameObject. Override Update() to print 'Player moving'.
- 2 Create a PowerUp class inheriting GameObject with an Activate() method.
- 3 Add base.Update(dt) inside Coin.Update() and print a message there — verify both run.
- 4 Add an abstract method 'GetDescription()' to GameObject. Make Coin and Enemy implement it.
- 5 Create a method PrintScene(List<GameObject>) that prints every object using ToString().

■ Quiz

Q1. What does 'class Coin : GameObject' mean?

- A. Coin contains a GameObject
- ✓ Coin inherits from GameObject
- C. Coin replaces GameObject
- D. Coin is a copy of GameObject

Answer: B

Q2. What does virtual mean on a method?

- A. Cannot be overridden
- B. Children must override it
- ✓ Children may override it
- D. Method has no body

Answer: C

Q3. What does override mean?

- A. Makes a new unrelated method
- ✓ Replaces the parent's virtual method
- C. Creates a copy
- D. Hides the parent

Answer: B

Q4. What is polymorphism in this context?

- A. All same type
- ✓ One list holding different types and calling the right method on each
- C. Hiding data
- D. Multiple constructors

Answer: B

Q5. What does `': base(x, y)'` do in a child constructor?

✓ Calls the base class constructor

- B. Creates a copy
- C. Sets static fields
- D. Calls `base.Update()`

Answer: A

LESSON

4

Interfaces

Contracts any class can fulfill — without deep hierarchies

■ Concept

An **interface** is a set of method signatures that a class promises to implement. Unlike inheritance (one parent only), a class can implement **any number of interfaces**. This lets you add cross-cutting behaviours — `ICollidable`, `IDrawable`, `IResettable` — without creating a deep inheritance chain.

Interfaces have no implementation — they are pure contracts. The compiler enforces that every method in the interface is provided by the implementing class. Interface names in C# conventionally start with 'I': `IDrawable`, `ICollidable`, `IInteractable`.

In your engine, interfaces are how you separate concerns. The collision system only knows about `ICollidable` — it does not care whether the object is a `Player`, `Enemy`, or `Coin`. The render system only knows about `IDrawable`. This keeps each system independent.

■ Project Setup — Add interface files to Lesson01

■ CREATE FILE

`ICollidable.cs`

Interface that any collidable object must implement.

■ CREATE FILE

`IResettable.cs`

Interface for objects that can be reset to their starting state.

■ CREATE FILE

`Program.cs`

Demonstrates filtering lists by interface type.

1

Define two interfaces

`ICollidable.cs`

Create **`ICollidable.cs`**. Interfaces contain only signatures — no method bodies.

```
ICollidable.cs
// ICollidable.cs
public interface ICollidable
{
    // Properties that must be provided – no implementation here
    float X { get; }
    float Y { get; }
    float Width { get; }
    float Height { get; }

    // Method that must be provided – also no body
    void OnCollision(GameObject other);
}

// IResettable.cs – put in its own file or add below
public interface IResettable
{
    void Reset();
}
```

■ Build & Run

\$ dotnet build

Expected output:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

■ Tip

An interface cannot have a constructor, fields, or any implementation. It only declares what must exist.

2

Implement interfaces in Player and Enemy

Enemy.cs

Update **Enemy.cs** to implement **ICollidable**. Add the required members. Also update **Player** with **ICollidable** and **IResettable**.



Enemy.cs

C#

```
// Enemy.cs – add interfaces after 'Enemy'
public class Enemy : GameObject, ICollidable
{
    // ... existing private fields and constructor stay the same ...

    // ICollidable requires these – compiler enforces them
    public float Width => 28f;
    public float Height => 36f;

    public void OnCollision(GameObject other)
    {
        Console.WriteLine($"Enemy hit by {other.GetType().Name}");
    }
    // ... TakeDamage and properties stay the same ...
}

// Add to Player.cs similarly:
// public class Player : GameObject, ICollidable, IResettable
// {
//     public float Width => 32f;
//     public float Height => 48f;
//     public void OnCollision(GameObject other) { TakeDamage(10); }
//     public void Reset() { _health = 100; _ammo = 10; }
// }
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)
0 Error(s)

3

Filter and use interfaces in Program.cs

Program.cs

Replace Program.cs to show how interfaces let you work with mixed object types through a shared contract.



Program.cs

C#

```
// Program.cs – Lesson 4
var scene = new List<GameObject>
{
    new Player(0, 0),
    new Enemy(100, 200, 50, 80f),
    new Coin(150, 180, 10),    // does NOT implement ICollidable
};

// OfType<T>() filters to only objects implementing that interface
var collidables = scene.OfType<ICollidable>().ToList();
Console.WriteLine($"Collidable objects: {collidables.Count}"); // 2

// Check each pair for collision (simplified)
for (int i = 0; i < collidables.Count; i++)
for (int j = i+1; j < collidables.Count; j++)
{
    var a = collidables[i]; var b = collidables[j];
    // Simple overlap check (AABB – covered properly in Lesson 14)
    bool overlap = a.X < b.X + b.Width && a.X + a.Width > b.X &&
        a.Y < b.Y + b.Height && a.Y + a.Height > b.Y;
    if (overlap)
        a.OnCollision((GameObject)b);
}

// IResettable filtering
var resettables = scene.OfType<IResettable>().ToList();
Console.WriteLine($"Resettable objects: {resettables.Count}"); // 1 (Player)
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
Collidable objects: 2
Resettable objects: 1
```

■ Tip

`OfType<T>()` returns only objects that implement the given interface or class. Objects that don't implement it are silently skipped — no cast exception.

➡ Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Create `IInteractable` with a string `Prompt` property and void `Interact()` method.
- 2 Create a `Door` class implementing `IInteractable`. `Interact()` prints 'Door opened!'.
- 3 Add `Door` to the scene list. Use `OfType<IInteractable>()` to find all interactables.
- 4 Call `Interact()` on each found interactable. Verify only the `Door` responds.
- 5 Try adding a `Coin` to `IInteractable` — add it, implement `Interact()`, and verify it is found too.

■ Quiz

Q1. How many interfaces can a class implement?

- A. One
- B. Two
- ☒ C. Unlimited
- D. None

Answer: C

Q2. What does an interface contain?

- A. Full implementations
- ☒ B. Method signatures only — no bodies
- C. Private fields
- D. Constructors

Answer: B

Q3. What does `OfType<ICollidable>()` do?

- A. Converts types
- ☒ B. Filters to only objects implementing `ICollidable`
- C. Sorts by type
- D. Removes nulls

Answer: B

Q4. Interface names in C# convention start with?

- A. _
- ☒ B. I
- C. C
- D. Abstract

Answer: B

Q5. Can an interface have fields?

- A. Yes
- B. Only static fields
- ✓ No — only method/property signatures**
- D. Only const fields

Answer: C

LESSON

5

Generics — List<T> and Dictionary<K,V>

Type-safe collections for managing engine objects and assets

■ Concept

Generics let you write code that works with any type while remaining type-safe. **List<T>** is a resizable array — you can add and remove items freely and it grows automatically. **Dictionary<K,V>** maps a key to a value with O(1) average lookup time — perfect for looking up textures, sounds, and scenes by name.

These two collections appear in virtually every engine system: the scene object list, the texture cache, the sound effect cache, the tile map, the input binding table, and more. Understanding them deeply now means every future lesson comes naturally.

■ Project Setup — Continue in Lesson01 — update Program.cs only

■ CREATE FILE

Program.cs

Replace with each step's code.

1

Work with List<T>

Program.cs

Replace **Program.cs** with this to explore List operations used throughout engine code.

```

Program.cs
// Program.cs – Lesson 5, List<T>
// Create a typed list – only Enemy objects allowed
List<Enemy> enemies = new List<Enemy>
{
    new Enemy(60, 80f),
    new Enemy(40, 120f),
    new Enemy(100, 50f),
};

// Count, access by index, iterate
Console.WriteLine($"Count: {enemies.Count}"); // 3
Console.WriteLine($"First HP: {enemies[0].Health}"); // 60

foreach (var e in enemies)
    Console.WriteLine(e.ToString());

// Add and remove
enemies.Add(new Enemy(80, 90f));
Console.WriteLine($"After Add: {enemies.Count}"); // 4

enemies.RemoveAt(0); // remove by index
Console.WriteLine($"After RemoveAt: {enemies.Count}"); // 3

// RemoveAll – safe removal by condition
enemies.RemoveAll(e => e.Health < 50);
Console.WriteLine($"After RemoveAll (<50 HP): {enemies.Count}");

// LINQ – filtering and sorting
var alive = enemies.Where(e => e.IsAlive).ToList();
var bySpeed = enemies.OrderByDescending(e => e.Speed).ToList();
Console.WriteLine($"Alive enemies: {alive.Count}");

```

■ Build & Run

\$ dotnet run

Expected output:

```

Count: 3
First HP: 60
Enemy at (0,0) alive=True
... (more)
After Add: 4
After RemoveAt: 3
After RemoveAll (<50 HP): 2
Alive enemies: 2

```

■ Tip

List<T>[i] access is O(1). List<T>.Contains() is O(n). For large collections that need fast lookup, use Dictionary<K,V> instead.

Replace **Program.cs** with this. A Dictionary is used in every engine to cache textures, sounds, and scenes by name.

```
Program.cs
// Program.cs – Lesson 5, Dictionary<K,V>
// String -> int mapping (like a score or points table)
var itemPoints = new Dictionary<string, int>
{
    { "coin", 10 },
    { "gem", 50 },
    { "trophy", 200 },
};

// O(1) lookup by key
Console.WriteLine($"coin = {itemPoints["coin"]}"); // 10
Console.WriteLine($"gem = {itemPoints["gem"]}"); // 50

// Safe lookup – TryGetValue avoids KeyNotFoundException
if (itemPoints.TryGetValue("boss_gem", out int pts))
    Console.WriteLine($"boss_gem = {pts}");
else
    Console.WriteLine("boss_gem not found – no crash!");

// Add and remove
itemPoints["star"] = 100; // add new key
itemPoints["coin"] = 15; // update existing key
itemPoints.Remove("trophy");

// Iterate key-value pairs
foreach (var kvp in itemPoints)
    Console.WriteLine($" {kvp.Key}: {kvp.Value} pts");

// ContainsKey check
Console.WriteLine($"Has 'gem': {itemPoints.ContainsKey("gem")}");
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
coin  = 10
gem   = 50
boss_gem not found – no crash!
  gem: 50 pts
  star: 100 pts
  coin: 15 pts
Has 'gem': True
```

■ Warning

itemPoints["missing_key"] throws KeyNotFoundException if the key does not exist. Always use TryGetValue() when the key might not be present.

3

Build a simple asset cache using Dictionary

Program.cs

Replace **Program.cs** to simulate the texture cache pattern your engine will use.

```
Program.cs C#
// Program.cs – Lesson 5, Asset Cache Pattern
// In a real engine this stores SDL_Texture* or Texture2D
// Here we use string to simulate the asset handle
var textureCache = new Dictionary<string, string>();

// Simulate loading textures (load each asset only once)
void LoadTexture(string name)
{
    if (textureCache.ContainsKey(name))
    { Console.WriteLine($" {name} already loaded – skip"); return; }
    textureCache[name] = $"<texture handle for {name}>";
    Console.WriteLine($"  Loaded: {name}");
}

LoadTexture("player");
LoadTexture("enemy");
LoadTexture("player"); // duplicate – skipped
LoadTexture("coin");
Console.WriteLine($"Cache size: {textureCache.Count}"); // 3

// Use a texture
if (textureCache.TryGetValue("player", out var tex))
    Console.WriteLine($"Drawing with: {tex}");
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
Loaded: player
Loaded: enemy
player already loaded – skip
Loaded: coin
Cache size: 3
Drawing with: <texture handle for player>
```

■ Tip

This exact pattern — load once into a Dictionary, look up by name — is used by ContentManager in MonoGame and AssetDatabase in Unity.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Create List<int> of scores: [45, 92, 17, 88, 63]. Print min, max, and average using LINQ.
- 2 Add 3 more scores and sort the list descending. Print the top 3 using .Take(3).
- 3 Create Dictionary<string, int> mapping enemy names to their health values.
- 4 Write a method LookupEnemy(string name) that uses TryGetValue and prints health or 'not found'.
- 5 Combine both: a List<string> of active enemy names, and a Dictionary for their stats. Print stats for each active enemy.

■ Quiz

Q1. What is the average lookup complexity of Dictionary<K,V>?

- A. $O(n)$
- B. $O(\log n)$
- ✓ C. $O(1)$
- D. $O(n^2)$

Answer: C

Q2. Why use TryGetValue instead of dict[key]?

- A. Faster
- ✓ B. Returns false instead of throwing when key missing
- C. Required by C#
- D. Thread-safe

Answer: B

Q3. List<T>.RemoveAll(predicate) removes?

- A. First match
- B. Last match
- ✓ **All matching items**
- D. Items at odd indices

Answer: C

Q4. What does LINQ .Where(e => e.IsAlive) return?

- A. Modifies the list
- ✓ **A new filtered enumerable**
- C. A bool
- D. The count

Answer: B

Q5. kvp.Key and kvp.Value are from?

- A. Array access
- ✓ **Iterating a Dictionary with foreach**
- C. LINQ query
- D. List indexing

Answer: B

LESSON

6

Delegates and Events

Callback functions and event-driven communication between engine systems

■ Concept

A **delegate** is a type-safe function pointer — a variable that holds a reference to a method. You can store it, pass it around, and call it later. This is how engines decouple systems: the AudioManager does not need to know about Enemy — it just subscribes to Enemy.Died and plays a sound when it fires.

Events are delegates with a restriction: only the class that declares the event can raise it. Any outside code can subscribe (+) or unsubscribe (-). This is identical to Unity's OnCollisionEnter, Button.onClick, and every other callback in Unity's API.

C# provides built-in delegate shorthand: **Action<T>** is a void delegate taking T as a parameter. **Func<T,TResult>** returns TResult. Use these instead of declaring your own delegate type unless you need a specific named type.

■ Project Setup — Continue in Lesson01 — update files

■ CREATE FILE

Enemy.cs

Add an event that fires when the enemy dies.

■ CREATE FILE

Program.cs

Subscribe to the event from outside.

1

Add a Died event to Enemy

Enemy.cs

Open **Enemy.cs** and add the event declaration and raise call. Add inside the Enemy class body.



Enemy.cs

C#

```
// Add inside the Enemy class, near the top:

// Event – any code can subscribe; only Enemy can raise it
// Action<Enemy> means: a void method that receives an Enemy
public event Action<Enemy>? Died;

// Raise it inside TakeDamage when health reaches zero:
// (Add the Died?.Invoke(this); line inside your existing if block)
// if (_health <= 0)
// {
//     _isDestroyed = true;
//     Died?.Invoke(this); // <-- add this line
//     Console.WriteLine("Enemy destroyed!");
// }

// Full updated TakeDamage for reference:
public void TakeDamage(int amount)
{
    _health -= amount;
    Console.WriteLine($"Enemy took {amount} dmg. HP: {_health}");
    if (_health <= 0)
    {
        _isDestroyed = true;
        Died?.Invoke(this); // raise the event – ?. = null safe
        Console.WriteLine("Enemy destroyed!");
    }
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

■ Tip

Died?.Invoke(this) — the ?. is the null-conditional operator. It means: call Invoke only if Died is not null (i.e., someone has subscribed). Without it, you get `NullReferenceException` when no subscribers exist.

2

Subscribe to the event in Program.cs

Program.cs

Replace **Program.cs** with this to subscribe multiple listeners to the Died event.



Program.cs

C#

```
// Program.cs – Lesson 6
int score = 0;

var goblin = new Enemy(30, 80f);

// += adds a subscriber (listener) to the event
// Lambda syntax: (who) => { ... } is an anonymous method
goblin.Died += (who) =>
{
    score += 100;
    Console.WriteLine($"Score +100! Total: {score}");
};

// You can add multiple subscribers – all will be called
goblin.Died += (who) => Console.WriteLine("Playing death sound!");
goblin.Died += (who) => Console.WriteLine($"Spawning drop at ({who.X},{who.Y})");

Console.WriteLine("Attacking goblin...");
goblin.TakeDamage(15); // HP: 15 – event does NOT fire
goblin.TakeDamage(20); // HP: -5 – event fires, all 3 subscribers called

Console.WriteLine($"Final score: {score}");

// Action and Func shorthand
Action<int> printDamage = dmg => Console.WriteLine($"Dealt {dmg} dmg");
Func<int,bool> isCritical = dmg => dmg > 50;
printDamage(30);
Console.WriteLine($"Is 60 crit? {isCritical(60)}");
```

■ Build & Run

\$ dotnet run

Expected output:

```
Attacking goblin...
Enemy took 15 dmg. HP: 15
Enemy took 20 dmg. HP: -5
Score +100! Total: 100
Playing death sound!
Spawning drop at (0,0)
Enemy destroyed!
Final score: 100
Dealt 30 dmg
Is 60 crit? True
```

■ Warning

Always unsubscribe (-=) from events in cleanup code. A subscriber that keeps a reference to a dead object prevents garbage collection — a memory leak. This matters especially with scene transitions.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Create a Door class with an event 'Action OnOpen'. Fire it in an Open() method.
- 2 Subscribe two lambdas: one prints 'Door opened!', one prints 'Creak sound played'.
- 3 Remove the first subscriber with -= and verify only the second fires after that.
- 4 Add 'Action<int> OnHealthChanged' to Enemy. Fire it every time TakeDamage runs with the new HP.
- 5 Create a HUD class that subscribes to OnHealthChanged and prints 'HP bar updated to N'.

■ Quiz

Q1. What does Died?.Invoke(this) do if no one has subscribed?

- A. Crashes
- B. Throws NullReferenceException
- ✓ Does nothing — ?. is null-safe
- D. Calls itself

Answer: C

Q2. What is Action<Enemy>?

- A. A class
- ✓ A void delegate that receives one Enemy parameter
- C. An interface
- D. A generic list

Answer: B

Q3. How do you add a subscriber to an event?

- A. event.Add()
- ✓ event += method
- C. event.Subscribe()
- D. event = method

Answer: B

Q4. What happens when multiple subscribers are attached?

- A. Only the first runs
- B. Only the last runs
- ✓ All run in subscription order**
- D. Random one runs

Answer: C

Q5. Why should you -= from events in cleanup?

- A. Performance
- ✓ Prevents memory leaks — subscriber holds references**
- C. Required by C#
- D. Prevents duplicate calls

Answer: B

LESSON

7

Properties — get and set

Controlled access to class data with validation and computed values

■ Concept

A **property** looks like a public field from the outside but runs code internally on read (**get**) and write (**set**). This lets you validate values on assignment, trigger side effects, and compute values on demand — all while keeping the public API clean and simple.

Properties are used extensively in engine code. Unity's `Transform.position` is a property that triggers internal updates when set. MonoGame's `SpriteBatch` also uses properties to control rendering state. You will write dozens of them in your own engine.

■ Project Setup — Continue in Lesson01

■ CREATE FILE

Player.cs

Add properties with validation to the Player class.

■ CREATE FILE

Program.cs

Test that validation fires correctly.

1

Add validated properties to Player

`Player.cs`

Open **Player.cs** and replace or update it with this version that uses full properties with validation logic.



```
// Player.cs – with validated properties
public class Player : GameObject
{
    private int    _health;
    private int    _maxHealth;
    private int    _ammo;
    private float  _speed;

    public Player(float x, float y, int maxHealth = 100)
        : base(x, y)
    {
        _maxHealth = maxHealth;
        _health    = maxHealth; // start at full health
        _ammo      = 10;
        _speed     = 180f;
    }

    // Full property with validation on set
    public int Health
    {
        get => _health;
        set => _health = Math.Clamp(value, 0, _maxHealth);
    }

    // Ammo – cannot go below 0 or above 30
    public int Ammo
    {
        get => _ammo;
        set => _ammo = Math.Clamp(value, 0, 30);
    }

    // Auto-property – compiler generates the backing field
    public float Speed { get; set; } = 180f;

    // Computed get-only – calculated fresh on every read
    public bool IsAlive    => _health > 0;
    public bool IsArmed    => _ammo > 0;
    public float HealthPct => (float)_health / _maxHealth;

    // Init-only – settable only at object construction
    public string Name { get; init; } = "Player";

    public void TakeDamage(int dmg) => Health -= dmg;
    public void Heal(int amount)    => Health += amount;
    public bool Shoot() { if (!IsArmed) return false; Ammo--; return true; }
}
```

■ Build & Run

\$ dotnet build

Expected output:

```
Build succeeded.  
0 Warning(s)  
0 Error(s)
```

■ Tip

`Math.Clamp(value, min, max)` restricts value to `[min, max]`. This is cleaner than writing `if/else` range checks manually.

2

Test properties and validation in Program.cs

Program.cs

Replace **Program.cs** to verify that validation fires on every assignment.

```
Program.cs C#  
// Program.cs – Lesson 7  
// Init-only Name must be set at construction  
var player = new Player(0, 0) { Name = "Hero" };  
Console.WriteLine($"Name: {player.Name}");  
// player.Name = "Other"; // COMPILER ERROR – Name is init-only  
  
Console.WriteLine($"Health: {player.Health}"); // 100  
Console.WriteLine($"HealthPct: {player.HealthPct:P0}"); // 100%  
  
// Over-healing is clamped to max  
player.Health += 50; // would be 150 – clamped to 100  
Console.WriteLine($"After over-heal: {player.Health}"); // 100  
  
// Damage beyond 0 is clamped to 0  
player.TakeDamage(999);  
Console.WriteLine($"After 999 dmg: {player.Health}"); // 0  
Console.WriteLine($"Alive: {player.IsAlive}"); // False  
  
// Ammo: shooting when empty returns false  
Console.WriteLine($"Ammo: {player.Ammo}");  
for (int i = 0; i < 12; i++)  
{  
    bool fired = player.Shoot();  
    Console.WriteLine($"Shot {i+1}: {(fired ? "BANG" : "EMPTY")}");  
}
```


■ Build & Run

```
$ dotnet run
```

Expected output:

```
Name: Hero
Health: 100
HealthPct: 100%
After over-heal: 100
After 999 dmg: 0
Alive: False
Ammo: 10
Shot 1: BANG
... (9 more shots)
Shot 11: EMPTY
Shot 12: EMPTY
```

■ Tip

The set accessor runs every time you write to the property — even self-modifying writes like `player.Health += 50`. That expands to `player.Health = player.Health + 50`, which goes through the setter and is clamped.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add a 'Speed' property to Player that clamps between 50f and 400f.
- 2 Add a read-only 'StatusLabel' property returning 'Dead', 'Critical', 'Hurt', or 'Healthy' based on HP%.
- 3 Make Ammo fire an event 'Action<int> OnAmmoChanged' every time it changes.
- 4 Add { get; private set; } to IsDestroyed — verify it cannot be set from Program.cs.
- 5 Create a ReadOnlyPlayer wrapper class that only exposes get-only versions of all Player properties.

■ Quiz

Q1. What does a property setter let you do vs a public field?

A. Nothing different

✓ Run validation code on every assignment

C. Make it thread-safe

D. Make it a constant

Answer: B

Q2. Math.Clamp(150, 0, 100) returns?

- A. 150
- B. 0
- ✓ 100**
- D. Throws

Answer: C

Q3. What does 'init' mean on a property?

- A. Always read-only
- ✓ Set once at construction or initialiser, then read-only**
- C. Auto-computed
- D. Thread-safe

Answer: B

Q4. 'player.Health += 50' calls?

- A. Neither get nor set
- B. Only get
- ✓ Get then set (expands to: Health = Health + 50)**
- D. Only arithmetic

Answer: C

Q5. Auto-property '{get; set;}' does?

- A. Nothing extra
- ✓ Compiler generates a private backing field automatically**
- C. Makes property static
- D. Prevents null

Answer: B

LESSON

8

Enums and Switch Expressions

Named constants and pattern matching for game state logic

■ Concept

An **enum** is a set of named integer constants. Instead of using magic numbers (`state == 2`), you write `state == GameState.Playing` — readable, refactorable, and impossible to accidentally assign an invalid value. Enums are used throughout engine code: game states, directions, animation states, item types, and collision layers.

C# **switch expressions** (available from C# 8) return a value and warn at compile time if you forget a case. The `_` pattern is the default. **[Flags] enums** use powers of two so multiple values can be combined with `|` — this is how Unity's `LayerMask` works.

■ Project Setup — Continue in Lesson01 — update Program.cs only

■ CREATE FILE

GameState.cs

New enum file.

■ CREATE FILE

Program.cs

Uses the enums.

1

Create enums for game states and directions

GameState.cs

Create **GameState.cs** with several enum declarations.

```

GameState.cs
// GameState.cs
// Simple enum – underlying type is int by default
public enum GameState
{
    MainMenu,    // = 0
    Playing,     // = 1
    Paused,      // = 2
    GameOver,    // = 3
}

public enum Direction { Left, Right, Up, Down, None }

public enum AnimState { Idle, Walk, Run, Jump, Fall, Attack, Die }

// [Flags] enum – values are powers of 2 so they can be ORed together
[Flags]
public enum CollisionLayer
{
    None        = 0,
    Ground      = 1,    // 0001
    Player      = 2,    // 0010
    Enemy       = 4,    // 0100
    Bullet      = 8,    // 1000
    All         = 15,   // 1111
}

```

■ Build & Run

```
$ dotnet build
```

Expected output:

```

Build succeeded.
    0 Warning(s)
    0 Error(s)

```

■ Warning

Every value in a [Flags] enum must be a power of 2. If you use 1,2,3,4 instead of 1,2,4,8, the value 3 is ambiguous — it could mean '1 and 2' OR just '3'.

2

Use switch expressions and flags in Program.cs

Program.cs

Replace **Program.cs** to demonstrate switch expressions and bitwise flag operations.



```
// Program.cs – Lesson 8
// Switch expression – maps enum to a value
GameState state = GameState.Playing;

string label = state switch
{
    GameState.MainMenu => "Press any key to start",
    GameState.Playing   => "PLAYING",
    GameState.Paused    => "PAUSED – press ESC to resume",
    GameState.GameOver  => "GAME OVER – press R to restart",
    _                   => "Unknown state"
};
Console.WriteLine($"State label: {label}");

// Switch expression returning an int
AnimState anim = AnimState.Jump;
int animFrames = anim switch
{
    AnimState.Idle      => 4,
    AnimState.Walk      => 8,
    AnimState.Run       => 8,
    AnimState.Jump      => 6,
    AnimState.Fall      => 4,
    AnimState.Attack    => 10,
    AnimState.Die       => 12,
    _                   => 1
};
Console.WriteLine($"{anim} has {animFrames} frames");

// [Flags] bitwise operations
CollisionLayer playerLayer = CollisionLayer.Player;
CollisionLayer mask = CollisionLayer.Player | CollisionLayer.Enemy;

bool playerInMask = (mask & CollisionLayer.Player) != 0;
bool bulletInMask = (mask & CollisionLayer.Bullet) != 0;
Console.WriteLine($"Player in mask: {playerInMask}"); // True
Console.WriteLine($"Bullet in mask: {bulletInMask}"); // False

// Casting enum to int and back
int raw = (int)GameState.GameOver;
Console.WriteLine($"GameOver as int: {raw}"); // 3
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
State label: PLAYING
Jump has 6 frames
Player in mask: True
Bullet in mask: False
GameOver as int: 3
```

■ Tip

Switch expressions produce a compile warning if you add a new enum value and forget to add a case for it. This is a major advantage over if/else chains — the compiler helps you stay exhaustive.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add a new enum `WeaponType { Sword, Bow, Staff, None }`. Write a switch expression mapping each to its damage value.
- 2 Add `EnemyType { Patrol, Chaser, Shooter, Boss }`. Write a method `GetSpeed(EnemyType)` using switch expression.
- 3 Create a [Flags] enum `ItemFlags { None=0, Stackable=1, Consumable=2, Questitem=4 }`. Combine `Stackable|Consumable` a
- 4 Cast `GameState` to `int` and back. Verify all four values round-trip correctly.
- 5 Rewrite a long if/else chain in your existing code using a switch expression.

■ Quiz

Q1. Main advantage of enum over magic numbers?

A. Faster

✓ **Named — readable and type-safe, impossible to use invalid value**

C. Less memory

D. Serialisable

Answer: B

Q2. What does `_` mean in a switch expression?

A. Match null

B. Match zero

✓ **Default — matches anything not covered**

D. Match empty string

Answer: C

Q3. Why must [Flags] values be powers of 2?

A. Compiler requirement

✓ **So OR-combined values don't create ambiguous overlap**

C. Performance

D. Style

Answer: B

Q4. Switch expression warns when?

A. Never

B. At runtime

✓ **A case is missing without _ fallback**

D. Always

Answer: C

Q5. (mask & CollisionLayer.Player) != 0 checks?

A. Equality

✓ **Whether Player bit is set in mask**

C. Whether mask equals Player

D. Whether both are zero

Answer: B

PART 2

2D ENGINE SYSTEMS

Game Loop · Rendering · Input · Physics · Audio · State Machine

LESSON

9

The Game Loop and Delta Time

The heartbeat of every game engine — input, update, draw, repeat

■ Concept

The **game loop** runs continuously — processing input, updating game state, and drawing to screen — many times per second. Every game engine ever written has this same fundamental structure. The only differences are what happens inside each step.

Delta time (dt) is the number of seconds since the last frame. Multiplying ALL movement, animation, and physics by dt makes your game run at the same speed on any hardware. Without it, a 240 FPS machine moves objects 4× faster than a 60 FPS machine — the game becomes unplayable on fast machines.

Always cap dt at a maximum (e.g. 0.05 seconds). Without a cap, if the game freezes for a second due to loading, dt = 1.0 and every object teleports a full second's worth of movement in one frame. This is called the **spiral of death** — large dt causes large moves which cause more lag which causes larger dt.

■ Project Setup — Create a new MonoGame project for the engine

```
dotnet new console -n Engine2D
cd Engine2D
dotnet add package MonoGame.Framework.DesktopGL
dotnet run
```

Terminal

■ CREATE FILE

Game1.cs

MonoGame's main game class — extends Game. Already exists after template creation.

■ CREATE FILE

Program.cs

Entry point — already exists. You will not modify this.

1

Understand the MonoGame game loop

Game1.cs

Open **Game1.cs** — MonoGame already provides it. Study the structure carefully. The comments below explain every part.

```
Game1.cs C#
// Game1.cs - MonoGame template with comments added
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

public class Game1 : Game
{
    private GraphicsDeviceManager _graphics;
    private SpriteBatch _spriteBatch;

    public Game1()
    {
        _graphics = new GraphicsDeviceManager(this);
        Content.RootDirectory = "Content";
        IsMouseVisible = true;
    }

    // Called once at startup - load assets here
    protected override void LoadContent()
    { _spriteBatch = new SpriteBatch(GraphicsDevice); }

    // Called every frame BEFORE Draw - game logic goes here
    // gameTime.ElapsedGameTime.TotalSeconds = dt
    protected override void Update(GameTime gameTime)
    {
        if (GamePad.GetState(PlayerIndex.One).Buttons.Back == ButtonState.Pressed
            || Keyboard.GetState().IsKeyDown(Keys.Escape))
            Exit();

        base.Update(gameTime); // always call base
    }

    // Called every frame AFTER Update - rendering goes here
    protected override void Draw(GameTime gameTime)
    {
        GraphicsDevice.Clear(Color.CornflowerBlue);
        base.Draw(gameTime); // always call base
    }
}
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(A window opens with a blue background – press Escape to close)

■ Warning

`base.Update(gameTime)` and `base.Draw(gameTime)` must always be called — MonoGame uses them internally for input processing and other housekeeping.

2

Add a moving ball using delta time

Game1.cs

Replace Game1.cs with this version that demonstrates delta time correctly. A ball moves across the screen at a fixed speed regardless of frame rate.



```
// Game1.cs - with delta time ball
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

public class Game1 : Game
{
    private GraphicsDeviceManager _graphics;
    private SpriteBatch _spriteBatch;
    private Texture2D _pixel;    // 1x1 white texture for drawing

    // Ball state
    private float _ballX = 100f;
    private float _ballY = 200f;
    private const float BALL_SPEED = 200f; // pixels per second

    // FPS counter
    private float _fpsTimer = 0f;
    private int _frameCount = 0;

    public Game1()
    {
        _graphics = new GraphicsDeviceManager(this);
        _graphics.PreferredBackBufferWidth = 800;
        _graphics.PreferredBackBufferHeight = 600;
        Content.RootDirectory = "Content";
        IsMouseVisible = true;
    }

    protected override void LoadContent()
    {
        _spriteBatch = new SpriteBatch(GraphicsDevice);
        // Create a 1x1 white pixel texture for drawing shapes
        _pixel = new Texture2D(GraphicsDevice, 1, 1);
        _pixel.SetData(new[] { Color.White });
    }

    protected override void Update(GameTime gameTime)
    {
        if (Keyboard.GetState().IsKeyDown(Keys.Escape)) Exit();

        // DELTA TIME - seconds since last frame
        float dt = (float)gameTime.ElapsedGameTime.TotalSeconds;
        dt = Math.Min(dt, 0.05f); // cap: prevent spiral of death

        // CORRECT: multiply by dt - same speed at any FPS
    }
}
```

```

Game1.cs (cont.)
_ballX += BALL_SPEED * dt;
// WRONG would be: _ballX += BALL_SPEED;

// Wrap ball when it leaves the right edge
if (_ballX > 800) _ballX = -20;

// FPS counter
_fpsTimer += dt; _frameCount++;
if (_fpsTimer >= 1f)
{
    System.Console.WriteLine($"FPS: {_frameCount}");
    _fpsTimer = 0f; _frameCount = 0;
}

base.Update(gameTime);
}

protected override void Draw(GameTime gameTime)
{
    GraphicsDevice.Clear(Color.CornflowerBlue);
    _spriteBatch.Begin();
    // Draw the ball as a 20x20 yellow square
    _spriteBatch.Draw(_pixel,
        new Rectangle((int)_ballX, (int)_ballY, 20, 20),
        Color.Yellow);
    _spriteBatch.End();
    base.Draw(gameTime);
}
}

```

■ Build & Run

\$ dotnet run

Expected output:

(Window opens, yellow square moves right at 200px/s – wraps back)
FPS: 60 (printed to console each second)

■ Tip

The ball moves at exactly 200 pixels per second regardless of frame rate. Try setting `IsFixedTimeStep=false` to uncap FPS and verify the speed stays the same.

3

Add a falling ball with gravity

Game1.cs

Add a vertical velocity variable and gravity to the existing **Game1.cs** to see delta-time physics working.



Game1.cs

C#

```
// Add these fields near the top of Game1 class:
private float _ballVY = 0f;           // vertical velocity
private const float GRAVITY = 400f;    // pixels per second squared

// Inside Update(), after _ballX line, add:
// Apply gravity – accelerates ball downward each frame
_ballVY += GRAVITY * dt;
_ballY  += _ballVY * dt;

// Bounce when ball hits the bottom of the window
if (_ballY > 580)
{
    _ballY = 580;
    _ballVY = -_ballVY * 0.8f; // lose 20% energy on bounce
}
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Ball now falls, accelerates, and bounces realistically)

■ Tip

This is exactly how all physics in a 2D platformer works — velocity accumulates gravity each frame. The dt multiplication is what makes the physics frame-rate independent.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add left/right ball control: pressing A/D adds horizontal velocity. Cap horizontal speed at 300.
- 2 Make the ball reset to the top-left when Space is pressed.
- 3 Add a second ball starting at a different position. Both should obey gravity independently.
- 4 Print the dt value to the console every frame. Notice it hovers around 0.016 at 60 FPS.
- 5 Set `IsFixedTimeStep=false` in the constructor. Observe the FPS counter change. Verify ball speed is unchanged.

■ Quiz

Q1. Why multiply movement by dt?

A. Limits top speed

✓ **Makes movement frame-rate independent**

C. Reduces CPU usage

D. Improves rendering

Answer: B

Q2. What is ElapsedGameTime.TotalSeconds?

A. Total game time

✓ **Time this frame took in seconds**

C. Time since last draw

D. FPS

Answer: B

Q3. Correct game loop order?

A. Draw then Update

B. Input then Draw then Update

✓ **Input then Update then Draw**

D. Update then Input then Draw

Answer: C

Q4. What does the dt cap (0.05f) prevent?

A. High FPS

✓ **Spiral of death from lag spikes**

C. Memory leaks

D. Frame tearing

Answer: B

Q5. base.Update(gameTime) must always be called because?

A. Style requirement

✓ **MonoGame uses it internally for input and housekeeping**

C. It draws to screen

D. It advances the timer

Answer: B

LESSON

10

Sprite Rendering with SpriteBatch

Loading textures and drawing sprites with MonoGame

■ Concept

A **Texture2D** is an image loaded into GPU memory. You load it once at startup and then draw it many times per frame at no extra cost — the data is already on the GPU. Load textures in **LoadContent()**, never in **Update()** or **Draw()**.

A **SpriteBatch** collects all draw calls and submits them to the GPU in one efficient batch at **End()**. Always call **Begin()** before any **Draw()** calls and **End()** when finished. Forgetting **End()** causes a crash on the next **Begin()**.

A **source Rectangle** crops a region from a sprite sheet — a single image file containing many animation frames side by side. This reduces GPU texture-switch overhead and is how all 2D game animation works.

■ Project Setup — Continue in the Engine2D MonoGame project from Lesson 9

■ CREATE FILE

Content/Content.mgcb

Open this in the MGCB Editor to add image assets.

■ CREATE FILE

Game1.cs

Updated to draw sprites.

1

Add a texture to the Content Pipeline

`Content/Content.mgcb`

Open the MGCB Editor (run 'dotnet tool install -g dotnet-mgcb-editor' then 'mgcb-editor' in the project folder). Add a PNG file. Alternatively, create a test texture in code:



Content/Content.mgcb

C#

```
// Game1.cs – Step 1: create a test texture in code
// (No PNG needed yet – we make a coloured texture from scratch)

// Add these fields:
private Texture2D _enemyTex;
private Texture2D _playerTex;

// In LoadContent(), add:
// Create a 32x32 red 'enemy' texture
_enemyTex = new Texture2D(GraphicsDevice, 32, 32);
Color[] red = new Color[32 * 32];
for (int i = 0; i < red.Length; i++) red[i] = Color.Red;
_enemyTex.SetData(red);

// Create a 32x48 blue 'player' texture
_playerTex = new Texture2D(GraphicsDevice, 32, 48);
Color[] blue = new Color[32 * 48];
for (int i = 0; i < blue.Length; i++) blue[i] = Color.SteelBlue;
_playerTex.SetData(blue);
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
```

■ Tip

In a real project, load PNG files with: `Texture2D tex = Content.Load<Texture2D>("myimage");` after adding them through the MGCB Editor.

2

Draw sprites with SpriteBatch

`Game1.cs`

Update **Draw()** in `Game1.cs` to draw the player and enemy textures at different positions.

```
Game1.cs C#
// In Draw(), replace GraphicsDevice.Clear line and add:
GraphicsDevice.Clear(Color.DarkSlateBlue);
_spriteBatch.Begin();

// Draw player at (100, 300) – full texture, no scaling
_spriteBatch.Draw(_playerTex, new Vector2(100, 300), Color.White);

// Draw enemy at (300, 300) – scaled to 64x64
Rectangle enemyDest = new Rectangle(300, 300, 64, 64);
_spriteBatch.Draw(_enemyTex, enemyDest, Color.White);

// Draw tinted (damaged) version
_spriteBatch.Draw(_enemyTex,
    new Rectangle(500, 300, 32, 32),
    Color.White * 0.4f); // 40% opacity

_spriteBatch.End();
```

■ Build & Run

\$ dotnet run

Expected output:

(Window shows blue player sprite, red enemy at normal size, faded red enemy)

■ Tip

`Color.White * 0.4f` multiplies the colour's alpha channel — this is how you do fade effects and transparency in MonoGame.

3

Draw with rotation — aiming a weapon

Game1.cs

In `Update()`, calculate the angle from the player to the mouse cursor. In `Draw()`, use the full `SpriteBatch.Draw` overload to rotate the sprite.

```

Game1.cs
C#
// Add field:
private float _playerAngle = 0f;

// In Update(), after the dt calculation:
MouseState ms = Mouse.GetState();
float dx = ms.X - (100 + 16); // 16 = half of 32px width (sprite centre)
float dy = ms.Y - (300 + 24); // 24 = half of 48px height
_playerAngle = MathF.Atan2(dy, dx);

// In Draw(), replace the player draw call with:
_spriteBatch.Draw(
    _playerTex,
    position:      new Vector2(100, 300),
    sourceRectangle: null,          // draw full texture
    color:         Color.White,
    rotation:      _playerAngle,   // in radians
    origin:        new Vector2(16, 24), // rotate around centre
    scale:         Vector2.One,
    effects:       SpriteEffects.None,
    layerDepth:    0f);

```

■ Build & Run

\$ dotnet run

Expected output:

(Player sprite rotates to always face the mouse cursor)

■ Warning

origin sets the pivot point. new Vector2(16, 24) is the centre of a 32x48 sprite. Without this, the sprite rotates around its top-left corner, which looks very wrong.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add a second enemy texture that is green. Draw 3 enemies at different positions.
- 2 Make the enemy sprite scale up when the mouse hovers over it (check distance from mouse to enemy position).
- 3 Draw a sprite at layerDepth 0.0 and another at 0.9. Use SpriteSortMode.BackToFront. Verify the 0.0 one draws on top.
- 4 Create a 4x1 sprite sheet (4 frames of 16px each) from code. Crop each frame using a source Rectangle and animate through.
- 5 Add a health bar: a red rectangle behind a green rectangle. Scale the green one by player.HealthPct.

■ Quiz

Q1. When should you load textures?

A. Every frame in Draw()

✓ **Once in LoadContent() — loading is slow**

C. Once per object

D. In Update()

Answer: B

Q2. What does SpriteBatch.End() do?

A. Clears the screen

✓ **Submits all batched draw calls to the GPU**

C. Resets the camera

D. Disposes textures

Answer: B

Q3. What does the source Rectangle do in Draw()?

A. Sets draw position

✓ **Crops a region from the texture — for sprite sheets**

C. Scales the texture

D. Sets rotation

Answer: B

Q4. What does 'origin' control in the full Draw overload?

A. Draw position

B. The texture file

✓ **The rotation and scale pivot point**

D. The colour

Answer: C

Q5. Why use a sprite sheet instead of separate image files?

A. Required

✓ **Reduces GPU texture switches — better performance**

C. Simpler to manage

D. Required by MonoGame

Answer: B

LESSON

11

Camera2D Transform Matrix

Scrolling the game world without moving any objects

■ Concept

A 2D camera works by applying an **offset matrix** to every sprite draw call. Instead of moving each object to the right when the camera moves left, you pass a single **transform matrix** to `SpriteBatch.Begin()` and the GPU handles all the offsetting automatically.

The matrix is composed of three operations: translate (move world by negative camera position), scale (apply zoom), and translate again (re-centre on screen). The camera smoothly follows the player using **linear interpolation (lerp)**: each frame, move 10% of the remaining distance to the target.

ScreenToWorld() converts a mouse click position (screen space) back to world space — essential for clicking on game objects when the camera is offset.

■ Project Setup — Add Camera2D.cs to the Engine2D project

■ CREATE FILE

Camera2D.cs

New file — the camera class.

■ CREATE FILE

Game1.cs

Updated to use the camera.

1

Create the Camera2D class

Camera2D.cs

Create **Camera2D.cs** in the Engine2D folder.



Camera2D.cs

C#

```
// Camera2D.cs
using Microsoft.Xna.Framework;

public class Camera2D
{
    public Vector2 Position { get; set; } = Vector2.Zero;
    public float Zoom { get; set; } = 1f;
    public float Rotation { get; set; } = 0f;

    private readonly GraphicsDeviceManager _gdm;
    public Camera2D(GraphicsDeviceManager gdm) => _gdm = gdm;

    private int ViewW => _gdm.PreferredBackBufferWidth;
    private int ViewH => _gdm.PreferredBackBufferHeight;

    // The transform matrix sent to SpriteBatch.Begin()
    public Matrix GetTransform()
    {
        return
            Matrix.CreateTranslation(-Position.X, -Position.Y, 0) *
            Matrix.CreateRotationZ(Rotation) *
            Matrix.CreateScale(Zoom, Zoom, 1f) *
            Matrix.CreateTranslation(ViewW / 2f, ViewH / 2f, 0);
    }

    // Smoothly lerp toward target – call from Update()
    public void Follow(Vector2 target, float speed, float dt)
    {
        Position = Vector2.Lerp(Position, target, speed * dt);
    }

    // Convert screen pixel position to world position
    public Vector2 ScreenToWorld(Vector2 screenPos)
    {
        return Vector2.Transform(screenPos,
            Matrix.Invert(GetTransform()));
    }

    // Clamp camera so it never shows outside the map
    public void Clamp(float mapW, float mapH)
    {
        float halfW = ViewW / (2f * Zoom);
        float halfH = ViewH / (2f * Zoom);
        Position = new Vector2(
            Math.Clamp(Position.X, halfW, mapW - halfW),
            Math.Clamp(Position.Y, halfH, mapH - halfH));
    }
}
```



Camera2D.cs (cont.)

C#

```
}
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Warning

The matrix order matters. Translate to origin first, then scale, then re-centre. Changing this order produces incorrect results.

2

Use the camera in Game1.cs

Game1.cs

Add a Camera2D field and hook it into Update() and Draw(). The player moves with arrow keys, the camera follows smoothly.


```

Game1.cs C#
// Add field:
private Camera2D _camera;
private Vector2 _playerPos = new Vector2(400, 300);

// In the Game1() constructor, after _graphics line:
// (we will initialise camera after _graphics is set up)

// In LoadContent(), add:
_camera = new Camera2D(_graphics);

// In Update(), add after dt calculation:
var kb = Keyboard.GetState();
float moveSpeed = 200f;
if (kb.IsKeyDown(Keys.Left)) _playerPos.X -= moveSpeed * dt;
if (kb.IsKeyDown(Keys.Right)) _playerPos.X += moveSpeed * dt;
if (kb.IsKeyDown(Keys.Up)) _playerPos.Y -= moveSpeed * dt;
if (kb.IsKeyDown(Keys.Down)) _playerPos.Y += moveSpeed * dt;

// Camera follows player, then clamps to world bounds
_camera.Follow(_playerPos, 8f, dt);
_camera.Clamp(2000, 1500); // world is 2000x1500 pixels

// Zoom with +/-
if (kb.IsKeyDown(Keys.OemPlus)) _camera.Zoom += 0.5f * dt;
if (kb.IsKeyDown(Keys.OemMinus)) _camera.Zoom -= 0.5f * dt;
_camera.Zoom = Math.Clamp(_camera.Zoom, 0.25f, 4f);

// In Draw(), pass the transform matrix to Begin():
_spriteBatch.Begin(transformMatrix: _camera.GetTransform());
// Draw player at world position
_spriteBatch.Draw(_playerTex, _playerPos, Color.White);
// Draw grid markers to show scrolling
for (int gx = 0; gx < 2000; gx += 100)
    for (int gy = 0; gy < 1500; gy += 100)
        _spriteBatch.Draw(_pixel,
            new Rectangle(gx-2, gy-2, 4, 4), Color.Gray);
_spriteBatch.End();

```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Arrow keys move player, camera follows. +/- zooms. Grid dots show world is large.)

■ Tip

The camera follow speed of 8f means: each second, close 8× the remaining distance to the target. At 60 FPS this gives very smooth, responsive following.

3

Add mouse world-position display

Game1.cs

Add this code to `Update()` to demonstrate `ScreenToWorld()` — essential for click-to-interact.

```
Game1.cs C#
// In Update(), add:
var ms = Mouse.GetState();
Vector2 mouseScreen = new Vector2(ms.X, ms.Y);
Vector2 mouseWorld = _camera.ScreenToWorld(mouseScreen);
Window.Title = $"Mouse world: ({mouseWorld.X:F0}, {mouseWorld.Y:F0})";
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Window title shows the world-space mouse position as you move it and scroll the camera)

■ Tip

`ScreenToWorld()` inverts the camera matrix to reverse the transformation. This is how you know which game object the player clicked on when the camera is scrolled or zoomed.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Make the camera rotate slowly using the camera's `Rotation` property. Notice how the world spins.
- 2 Add a minimap: draw all objects a second time in the corner with `Zoom=0.1f` and no camera offset.
- 3 Make the camera shake: when `Space` is pressed, add a random offset that decays to zero over 0.5 seconds.
- 4 Add map boundary walls — the player should not be able to leave the 2000x1500 world.
- 5 Print the current camera zoom in the window title alongside the mouse world position.

■ Quiz

Q1. What does the transform matrix in `SpriteBatch.Begin()` do?

- A. Clears screen
- ✓ Applies position and zoom to all sprite draws
- C. Sorts by depth
- D. Changes clear colour

Answer: B

Q2. Lerp speed 8f means?

- A. Move 8 pixels per frame
- ✓ Close 8x remaining distance per second**
- C. Snap instantly at 8 FPS
- D. 8 pixel offset

Answer: B

Q3. ScreenToWorld() is needed for?

- A. Rendering
- ✓ Converting mouse click position to world coordinates**
- C. Loading assets
- D. Sorting sprites

Answer: B

Q4. Why clamp the camera position?

- A. Performance
- ✓ Prevent showing empty space outside the map**
- C. Required by MonoGame
- D. Sorting

Answer: B

Q5. Matrix order for camera: translate then scale then retranslate — what breaks if you change order?

- A. Nothing
- ✓ The camera ends up in the wrong position**
- C. Performance drops
- D. Textures fail to load

Answer: B

LESSON

12

Input — Held vs Pressed vs Released

Reading keyboard and mouse correctly with precise per-frame state

■ Concept

Input exists in exactly three states per key per frame: **held** (the key is currently down — true every frame), **pressed** (the key went down this frame — true for exactly one frame), and **released** (the key came up this frame — true for exactly one frame).

MonoGame's `Keyboard.GetState()` gives you the current state. To get pressed/released, you compare it against the previous frame's state. This is why you keep a copy of the old state: `_prevKeyboard = _currKeyboard` at the start of every `Update()`.

Always call `Keyboard.GetState()` exactly once per frame and store it. Do not call it multiple times in the same frame — the result could differ between calls on some platforms.

■ Project Setup — Add `InputManager.cs` to `Engine2D`

■ CREATE FILE

`InputManager.cs`

The input manager class — one per game.

■ CREATE FILE

`Game1.cs`

Updated to use `InputManager`.

1

Create `InputManager.cs`

`InputManager.cs`

Create **`InputManager.cs`**. This class tracks current and previous keyboard/mouse state.

[illegible]

```
InputManager.cs (cont.)
=> _currM.ScrollWheelValue - _prevM.ScrollWheelValue;
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

2

Wire InputManager into Game1.cs and demonstrate all three states

Game1.cs

Add an InputManager field to Game1. Call Update() at the start of the game's Update(). Then demonstrate all three input states.

```
Game1.cs C#
// Add field:
private readonly InputManager _input = new InputManager();
private bool _toggleState = false;
private float _jumpVelocity = 0f;

// At the VERY START of Update(), before anything else:
_input.Update(); // must be first

// Then demonstrate the three states:
// IsHeld – movement (true every frame key is down)
if (_input.IsHeld(Keys.Right)) _playerPos.X += 200f * dt;
if (_input.IsHeld(Keys.Left)) _playerPos.X -= 200f * dt;

// IsPressed – fires exactly ONCE per key press
if (_input.IsPressed(Keys.Space))
{
    _jumpVelocity = -400f;
    Console.WriteLine("Jump!"); // prints once per press, not every frame
}

// IsPressed for toggle – flip on each press
if (_input.IsPressed(Keys.T))
{
    _toggleState = !_toggleState;
    Console.WriteLine($"Toggle: {_toggleState}");
}

// IsReleased – variable jump height
if (_input.IsReleased(Keys.Space))
    if (_jumpVelocity < -100f) _jumpVelocity *= 0.5f; // cut jump

// Apply jump velocity
_jumpVelocity += 600f * dt; // gravity
_playerPos.Y += _jumpVelocity * dt;
if (_playerPos.Y > 400) { _playerPos.Y = 400; _jumpVelocity = 0; }

// Scroll wheel zoom
int scroll = _input.ScrollDelta;
if (scroll != 0) _camera.Zoom += scroll * 0.001f;
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Arrow keys move player left/right. Space jumps. T toggles. Mouse wheel zooms.)

Jump! (printed once per spacebar press, not every frame)

Toggle: True

Toggle: False

■ Warning

`InputManager.Update()` must always be the very first call in `Game1.Update()`. If you call it after moving the player, the player sees last frame's input — subtle but causes a 1-frame input lag.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Add a dash ability: pressing Shift launches the player 150px in the current movement direction instantly.
- 2 Add double-jump: pressing Space while airborne (and having jumps remaining) should also jump.
- 3 Show 'HELD', 'PRESSED', or 'RELEASED' on-screen using `SpriteBatch` text when keys are active.
- 4 Make right-click set a target position and the player slowly moves toward it using `lerp`.
- 5 Add a simple key rebinding: pressing R enters 'rebind mode', the next key pressed becomes the jump key.

■ Quiz

Q1. `IsPressed` returns true for how many frames?

- A. Every frame
- B. Two
- ✓ C. Exactly one
- D. Zero

Answer: C

Q2. When should `InputManager.Update()` be called?

- A. After game logic
- ✓ B. Before any game logic — first in `Update()`
- C. In `Draw()`
- D. Once at startup only

Answer: B

Q3. `IsReleased` is useful for?

- A. First press detection
- B. Every frame held
- ✓ C. Just released — e.g. variable jump height, releasing charge attack
- D. Reset

Answer: C

Q4. Why keep a previous KeyboardState?

A. Performance

✓ **Needed to compute pressed/released (diff between frames)**

C. Required by MonoGame

D. Thread safety

Answer: B

Q5. What does ScrollDelta return?

A. Mouse position

✓ **How much the scroll wheel moved this frame**

C. Click count

D. Absolute scroll value

Answer: B

LESSON

13

Scene Manager

Owning, updating, and cleaning up all game objects safely

■ Concept

The **Scene** is the container that owns all active game objects. Each frame it calls `Update()` on every active object, then `Draw()` on every visible one, then removes destroyed objects. In Unity this is the Hierarchy panel — a list of `GameObjects` that the engine loops through every frame.

A critical detail: you cannot add or remove objects from a list while iterating it with `foreach`. If you add a new enemy during `Update()` (e.g., a boss spawning minions), it must go into a **pending additions buffer** and be flushed at the start of the next frame.

Objects are not immediately removed when `IsDestroyed` is set. They finish the current frame's `Update`, and are cleaned up at the end — this prevents skipping objects in the loop and avoids crashes.

■ Project Setup — Add `Scene.cs` and `GameObject.cs` to `Engine2D`

■ CREATE FILE

`Scene.cs`

The scene container class.

■ CREATE FILE

`Game1.cs`

Updated to use `Scene` instead of manual lists.

1

Create the Scene class

`Scene.cs`

Create **`Scene.cs`**. This replaces any manual `List` in your game.



Scene.cs

C#

```
// Scene.cs
using System.Collections.Generic;
using System.Linq;
using Microsoft.Xna.Framework.Graphics;

public class Scene
{
    // The main list – never modify during Update or Draw
    private readonly List<GameObject> _objects = new();

    // Pending additions – flushed at start of next Update
    private readonly List<GameObject> _toAdd = new();

    // Queue an object to be added next frame (safe during Update)
    public void Add(GameObject obj) => _toAdd.Add(obj);

    public void Update(float dt)
    {
        // 1. Flush pending additions
        if (_toAdd.Count > 0)
        { _objects.AddRange(_toAdd); _toAdd.Clear(); }

        // 2. Update all active objects
        foreach (var obj in _objects)
            if (obj.IsActive) obj.Update(dt);
    }

    public void Draw(SpriteBatch sb)
    {
        // Draw in Z-order (background first, foreground last)
        foreach (var obj in _objects.OrderBy(o => o.ZOrder))
            if (obj.IsActive) obj.Draw(sb);
    }

    // Remove destroyed objects – call at end of Update
    public void Cleanup()
        => _objects.RemoveAll(o => o.IsDestroyed);

    // Find all objects of a specific type
    public IEnumerable<T> GetAll<T>() where T : GameObject
        => _objects.OfType<T>();

    // Find first object of a specific type
    public T? Find<T>() where T : class
        => _objects.OfType<T>().FirstOrDefault();
}
```



Scene.cs (cont.)

C#

```
public int Count => _objects.Count;
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Tip

Always call `Cleanup()` at the end of `Update()` — after all objects have had a chance to update. Objects destroyed this frame are removed before the next frame's `Draw`.

2

Update GameObject to have ZOrder

`GameObject.cs`

Open **GameObject.cs** and add a `ZOrder` property for draw layering.

```
GameObject.cs
// Add to the GameObject class:
public int ZOrder { get; set; } = 0;
// Lower ZOrder = drawn earlier (behind)
// Higher ZOrder = drawn later (in front)

// Typical values:
// Background tiles: ZOrder = -100
// Enemies:          ZOrder = 0
// Player:           ZOrder = 1
// UI / HUD:         ZOrder = 100
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

3

Use Scene in Game1.cs

`Game1.cs`

Replace any manual object lists in `Game1` with a single `Scene`. Show how spawning works safely mid-update.

```

Game1.cs C#
// Add field:
private Scene _scene = new Scene();

// In LoadContent() - populate scene:
var player = new Player(400, 300);
player.ZOrder = 1;
_scene.Add(player);

for (int i = 0; i < 5; i++)
{
    var enemy = new Enemy(100 + i*80, 200, 50, 80f);
    enemy.ZOrder = 0;
    _scene.Add(enemy);
}

// In Update():
_input.Update();
float dt = (float)gameTime.ElapsedGameTime.TotalSeconds;
dt = Math.Min(dt, 0.05f);
_scene.Update(dt);
_scene.Cleanup();
Console.WriteLine($"Objects: {_scene.Count}");

// In Draw():
GraphicsDevice.Clear(Color.DarkSlateBlue);
_spriteBatch.Begin(transformMatrix: _camera.GetTransform());
_scene.Draw(_spriteBatch);
_spriteBatch.End();

```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Window shows player and 5 enemies)
Objects: 6 (printed each frame)

■ Tip

Find<Player>() returns the first Player in the scene. GetAll<Enemy>() returns all enemies as an IEnumerable. These are used in collision systems to efficiently find relevant objects.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Spawn a new Enemy from inside an existing Enemy's Update() using scene.Add(). Verify it appears next frame.
- 2 Make enemies destroy themselves when their health reaches 0. Verify Cleanup() removes them.
- 3 Add a 'Wave' system: when all enemies are destroyed, call a SpawnWave() method that adds 3 new ones.
- 4 Implement scene.Clear() that marks all objects as destroyed and calls Cleanup().
- 5 Add a Find<Player>() call and make all enemies in the scene move toward the player's position each frame.

■ Quiz

Q1. Why use a _toAdd pending buffer?

A. Performance

✓ Prevents collection-modified exception when adding during iteration

C. Required by MonoGame

D. Sorting

Answer: B

Q2. GetAll<Enemy>() returns?

A. All GameObjects

✓ Only Enemy instances

C. Only active enemies

D. Only first enemy

Answer: B

Q3. When should Cleanup() be called?

A. Start of Update

B. During Draw

✓ End of Update — after all logic runs

D. Every 10 frames

Answer: C

Q4. OrderBy(o => o.ZOrder) does?

A. Removes duplicates

✓ Sorts objects so lower ZOrder draws first

C. Filters inactive

D. Groups by type

Answer: B

Q5. Find<Player>() returns?

A. All players

✓ **The first Player in the scene**

C. Null always

D. A list

Answer: B

LESSON

14

AABB Collision Detection

Finding overlapping rectangles — the foundation of 2D collision

■ Concept

AABB (Axis-Aligned Bounding Box) collision requires the rectangles to overlap on **both** the X axis AND the Y axis simultaneously. If either axis has a gap, there is no collision. This is $O(1)$ per pair — fast enough to check hundreds of pairs per frame.

MonoGame's **Rectangle.Intersects()** implements exactly this check. You define bounding boxes for each object and test pairs. To keep performance manageable, only test objects that could plausibly overlap — use broad-phase filtering first.

Circle collision compares squared distances (avoiding the slow square root) and is more natural for round or freely-rotating objects.

■ Project Setup — Add collision checking to Engine2D

■ CREATE FILE

CollisionSystem.cs

Handles all collision checks — separated from GameObject.

■ CREATE FILE

Game1.cs

Calls the collision system each frame.

1

Add Bounds to GameObject and create CollisionSystem

`CollisionSystem.cs`

First, add a Bounds property to **GameObject.cs**. Then create **CollisionSystem.cs**.



```
// Add to GameObject.cs:
public virtual Rectangle Bounds
    => new Rectangle((int)X, (int)Y, 32, 32); // default 32x32

// CollisionSystem.cs
using Microsoft.Xna.Framework;
using System.Collections.Generic;

public static class CollisionSystem
{
    // Manual AABB check – both axes must overlap
    public static bool AABBOverlap(Rectangle a, Rectangle b)
        => a.Left < b.Right && a.Right > b.Left &&
           a.Top < b.Bottom && a.Bottom > b.Top;

    // MonoGame built-in (same logic, slightly faster)
    public static bool Overlap(Rectangle a, Rectangle b)
        => a.Intersects(b);

    // Circle collision – avoids slow Math.Sqrt
    public static bool CircleOverlap(
        Vector2 centreA, float rA,
        Vector2 centreB, float rB)
    {
        float dx = centreA.X - centreB.X;
        float dy = centreA.Y - centreB.Y;
        float rSum = rA + rB;
        return dx*dx + dy*dy <= rSum*rSum; // compare squared
    }

    // Check player vs all enemies in a scene
    public static void CheckPlayerVsEnemies(
        Player player, IEnumerable<Enemy> enemies)
    {
        Rectangle pb = player.Bounds;
        foreach (var enemy in enemies)
        {
            if (enemy.IsDestroyed) continue;
            if (pb.Intersects(enemy.Bounds))
            {
                player.TakeDamage(1);
                Console.WriteLine("Player hit!");
            }
        }
    }
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Tip

Only check collisions between objects that could plausibly overlap. If you have 100 enemies, checking every pair is 10,000 checks per frame. Use `GetAll<Enemy>()` to limit to just relevant types.

2

Visualise bounding boxes in Draw

Game1.cs

Add **debug rendering** to `Draw()` so you can see the collision boxes. This is invaluable for tuning.

```
Game1.cs C#
// Add a debug toggle field:
private bool _debugDraw = false;

// In Update(), toggle with D key:
if (_input.IsPressed(Keys.D)) _debugDraw = !_debugDraw;

// At the END of Draw(), inside Begin/End blocks:
if (_debugDraw)
{
    // Draw player bounds in green
    var pb = _scene.Find<Player>()?.Bounds ?? Rectangle.Empty;
    if (pb != Rectangle.Empty)
        DrawRect(pb, Color.Green * 0.5f);

    // Draw all enemy bounds in red
    foreach (var e in _scene.GetAll<Enemy>())
        DrawRect(e.Bounds, Color.Red * 0.5f);
}

// Helper method – add outside Draw():
private void DrawRect(Rectangle r, Color c)
{
    _spriteBatch.Draw(_pixel, new Rectangle(r.X, r.Y, r.Width, 1), c);
    _spriteBatch.Draw(_pixel, new Rectangle(r.X, r.Bottom-1, r.Width, 1), c);
    _spriteBatch.Draw(_pixel, new Rectangle(r.X, r.Y, 1, r.Height), c);
    _spriteBatch.Draw(_pixel, new Rectangle(r.Right-1, r.Y, 1, r.Height), c);
}
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Press D to toggle debug bounding boxes – green for player, red for enemies)

■ Tip

Always build debug visualisation for collision boxes. It is the fastest way to spot misaligned hitboxes, which are a very common source of frustration in 2D game development.

3

Run collision checks in Update

Game1.cs

Add to Update() after `_scene.Update(dt)` — after objects have moved to their new positions.

```
Game1.cs C#
// In Update(), after _scene.Update(dt):
var player = _scene.Find<Player>();
var enemies = _scene.GetAll<Enemy>();

if (player != null)
    CollisionSystem.CheckPlayerVsEnemies(player, enemies);
```

■ Build & Run

```
$ dotnet run
```

Expected output:

Player hit! (printed when player overlaps any enemy)

■ Warning

Run collision checks **AFTER** all objects have updated their positions. Running before means you are checking last frame's positions.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Override Bounds in Player to be 24x44 instead of 32x32 — a tighter hitbox that feels fairer.
- 2 Add coin collection: check Player vs all Coin objects. When overlapping, collect the coin (IsDestroyed=true, score++).
- 3 Implement enemy-vs-enemy collision: enemies should not overlap each other. Push them apart when they do.
- 4 Add a visualisation of the overlap amount: draw a small cross at the centre of each collision.
- 5 Optimise: only check enemies within 200 pixels of the player using distance-squared filtering first.

■ Quiz

Q1. AABB overlap requires?

- A. X axis overlap only
- B. Y axis overlap only
- ✓ Both X AND Y axes overlapping simultaneously
- D. Either axis overlapping

Answer: C

Q2. CircleOverlap uses $dx*dx+dy*dy$ because?

- A. Required
- ✓ Avoids slow square root — squared comparison is equivalent
- C. More accurate
- D. Easier to read

Answer: B

Q3. When should collision checks run?

- A. Before objects update
- B. During object Update()
- ✓ After all objects have updated positions
- D. During Draw()

Answer: C

Q4. Why override Bounds in Player to be smaller than the sprite?

- A. Required
- ✓ Tighter hitbox feels fairer — sprite edges are rarely visually meaningful
- C. Performance
- D. Required by CollisionSystem

Answer: B

Q5. GetAll<Enemy>() vs checking all scene objects?

- A. Same
- ✓ GetAll<Enemy>() only returns enemies — avoids checking coins, tiles, etc.
- C. GetAll is slower
- D. GetAll returns inactive too

Answer: B

LESSON

15

AABB Collision Resolution

Pushing objects apart after a collision is detected

■ Concept

Detecting a collision is only half the job. **Resolution** pushes overlapping objects apart so they don't pass through each other. The technique: find the overlap amount on both axes, then push on the axis with the **smallest overlap** — this is the minimum separation needed and produces the most natural-feeling result.

For tile-based worlds, always resolve the **Y axis first** (landing on top of tiles), then recheck and resolve the **X axis** (hitting walls). Doing them in the wrong order causes the player to stick to walls when trying to land.

After each push, **refresh the bounding box** — the object has moved, so subsequent tile checks must use the updated position.

■ Project Setup — Add resolution to CollisionSystem.cs

■ CREATE FILE

CollisionSystem.cs

Add GetOverlapPush and tile resolution methods.

■ CREATE FILE

TileMap.cs

Simple tile map for testing.

1

Add GetOverlapPush to CollisionSystem

CollisionSystem.cs

Open **CollisionSystem.cs** and add the overlap push calculation.



```
// Add to CollisionSystem.cs:

// Returns the minimum push vector to separate two overlapping rectangles
public static Vector2 GetOverlapPush(Rectangle a, Rectangle b)
{
    // Calculate overlap on both axes
    float ox = Math.Min(a.Right, b.Right) - Math.Max(a.Left, b.Left);
    float oy = Math.Min(a.Bottom, b.Bottom) - Math.Max(a.Top, b.Top);

    // Push on the SMALLER overlap axis — minimum separation
    if (ox < oy)
    {
        // X axis is smaller — push left or right
        float dir = a.Center.X < b.Center.X ? -1f : 1f;
        return new Vector2(ox * dir, 0);
    }
    else
    {
        // Y axis is smaller — push up or down
        float dir = a.Center.Y < b.Center.Y ? -1f : 1f;
        return new Vector2(0, oy * dir);
    }
}

// Returns whether the push was upward (player landed on tile top)
public static bool PushHitTop(Vector2 push) => push.Y < 0;
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Tip

`push.Y < 0` means the player was pushed upward — their bottom edge was overlapping a tile's top edge. This means they landed on the tile. Use this to set `grounded=true`.

2

Create a simple TileMap

TileMap.cs

Create **TileMap.cs** to represent the game world as a 2D grid.



TileMap.cs

C#

```
// TileMap.cs
using Microsoft.Xna.Framework;

public class TileMap
{
    public int TileSize { get; } = 32;
    private readonly int[,] _tiles;
    public int Width => _tiles.GetLength(1);
    public int Height => _tiles.GetLength(0);

    // 0 = empty, 1 = solid
    public TileMap(int[,] tiles) { _tiles = tiles; }

    public bool IsSolid(int tx, int ty)
    {
        if (tx < 0 || ty < 0 || tx >= Width || ty >= Height)
            return false; // out of bounds = not solid
        return _tiles[ty, tx] == 1;
    }

    public Rectangle TileBounds(int tx, int ty)
        => new Rectangle(tx * TileSize, ty * TileSize,
            TileSize, TileSize);

    // Get all solid tiles that might overlap a given rectangle
    public (int tx0, int ty0, int tx1, int ty1) GetNearbyRange(Rectangle r)
    {
        return (r.X/TileSize, r.Y/TileSize,
            (r.Right-1)/TileSize, (r.Bottom-1)/TileSize);
    }
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
```

3

Resolve player vs tile collisions

`CollisionSystem.cs`

Add tile resolution to `CollisionSystem.cs`. This is the core of platformer physics.

```

CollisionSystem.cs
// Add to CollisionSystem.cs:
public static bool ResolveTileCollision(
    ref float x, ref float y, int w, int h,
    TileMap map, out bool hitTop)
{
    hitTop = false;
    bool hit = false;
    Rectangle pb = new Rectangle((int)x, (int)y, w, h);

    var (tx0,ty0,tx1,ty1) = map.GetNearbyRange(pb);

    for (int ty = ty0; ty <= ty1; ty++)
    for (int tx = tx0; tx <= tx1; tx++)
    {
        if (!map.IsSolid(tx, ty)) continue;
        Rectangle tile = map.TileBounds(tx, ty);
        if (!pb.Intersects(tile)) continue;

        Vector2 push = GetOverlapPush(pb, tile);
        x += push.X; y += push.Y;
        if (PushHitTop(push)) hitTop = true;
        pb = new Rectangle((int)x, (int)y, w, h); // REFRESH
        hit = true;
    }
    return hit;
}

```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Player now lands on tiles and slides against walls – test in your Game1)

■ Warning

Refreshing pb after each push is essential. After the player is pushed out of one tile, they may now overlap a different tile. Using the old position for subsequent checks produces incorrect results.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add a platform that the player can fall through from below but land on from above (one-way platform).
- 2 Make enemies also collide with tiles so they don't walk off the edge.
- 3 Add a moving platform: a tile that moves horizontally. The player should ride it when standing on top.
- 4 Visualise the push vector with a coloured line in debug mode.
- 5 Add hazard tiles (type 2) that kill the player on contact without pushing them away.

■ Quiz

Q1. Why push on the smallest overlap axis?

- A. Required
- ✓ **Minimum movement — most natural separation**
- C. Faster
- D. Required for Y detection

Answer: B

Q2. $\text{push.Y} < 0$ means?

- A. Player hit a wall
- ✓ **Player was pushed upward — landed on tile top**
- C. Ceiling hit
- D. Gravity reversed

Answer: B

Q3. Why refresh the bounding box after each push?

- A. Required
- ✓ **Position changed — next tile check needs updated position**
- C. Performance
- D. Prevents NaN

Answer: B

Q4. Resolve Y before X for tiles because?

- A. Y is more important
- ✓ **Landing detection must work before wall detection — wrong order causes sticking**
- C. X is harder
- D. Required

Answer: B

Q5. What does GetNearbyRange() return?

A. All tiles

✓ **Tile index range that overlaps the given rectangle — avoids full map scan**

C. Random tiles

D. Solid tiles only

Answer: B

Physics Body — Velocity and Gravity

Realistic movement feel through velocity and acceleration

■ Concept

Game physics is not about accuracy — it is about **feel**. The goal is movement that feels responsive and satisfying, not realistic. Real-world gravity (9.8 m/s^2) feels horribly floaty in a game. Typical platformer gravity is 10-100× stronger.

Velocity is speed with direction. Each frame: $\text{position} += \text{velocity} * \text{dt}$. **Gravity** is a constant downward acceleration added to Y velocity every frame: $\text{vy} += \text{gravity} * \text{dt}$. **Terminal velocity** caps the maximum fall speed to prevent tunnelling through thin platforms at high speed.

Three values to tune: gravity, jump force, and friction. Tune them together — $\text{jump height} = \text{jumpForce}^2 / (2 \times \text{gravity})$. With $\text{gravity}=900$ and $\text{jumpForce}=420$, the player reaches about 98 pixels (3 tiles).

■ Project Setup — Add PhysicsBody.cs to Engine2D

■ CREATE FILE

PhysicsBody.cs

The physics component — attached to Player and enemies.

■ CREATE FILE

Player.cs

Updated to use PhysicsBody.

1

Create the PhysicsBody class

PhysicsBody.cs

Create **PhysicsBody.cs**. This encapsulates all movement physics.



PhysicsBody.cs

C#

```
// PhysicsBody.cs
using Microsoft.Xna.Framework;

public class PhysicsBody
{
    // Current velocity in pixels per second
    public Vector2 Velocity { get; set; } = Vector2.Zero;
    public float Gravity { get; set; } = 900f;
    public float MaxFallSpeed { get; set; } = 700f;
    public bool IsGrounded { get; set; }

    // Apply gravity and move – call from object's Update()
    public void Update(float dt, ref float x, ref float y)
    {
        // Apply gravity only when airborne
        if (!IsGrounded)
        {
            var v = Velocity;
            // Cap fall speed – prevents tunnelling
            v.Y = Math.Min(v.Y + Gravity * dt, MaxFallSpeed);
            Velocity = v;
        }
        x += Velocity.X * dt;
        y += Velocity.Y * dt;
    }

    // Apply an instant upward velocity – call on jump input
    public void Jump(float force)
    {
        // Negative Y = upward (screen Y increases downward)
        Velocity = new Vector2(Velocity.X, -force);
        IsGrounded = false;
    }

    // Gradually decelerate horizontal speed – call when no input
    public void ApplyFriction(float friction, float dt)
    {
        float vx = Velocity.X;
        // Reduce magnitude toward zero without overshooting
        vx = vx > 0 ? Math.Max(0, vx - friction * dt)
            : Math.Min(0, vx + friction * dt);
        Velocity = new Vector2(vx, Velocity.Y);
    }

    // Called by tile collision when player lands on tile top
    public void SetGrounded() { IsGrounded = true; Velocity = new Vector2(Velocity.X, 0); }
}
```



PhysicsBody.cs (cont.)

C#

```
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Tip

Velocity.Y is negative when moving up (jump) and positive when moving down (fall). This is because screen Y increases downward in MonoGame and most 2D engines.

2

Integrate **PhysicsBody** into **Player**

`Player.cs`

Open **Player.cs** and add a **PhysicsBody**. Call it from **Update**, resolve tiles, then set grounded.

```

Player.cs C#
// Add field to Player:
public readonly PhysicsBody Physics = new PhysicsBody();

// Player needs a reference to TileMap – add it:
private readonly TileMap _map;

// Update Player constructor to accept TileMap:
public Player(float x, float y, TileMap map) : base(x, y)
{ _map = map; }

// Override Update():
public override void Update(float dt)
{
    // Horizontal input
    float vx = 0;
    var kb = Microsoft.Xna.Framework.Input.Keyboard.GetState();
    if (kb.IsKeyDown(Microsoft.Xna.Framework.Input.Keys.Left)) vx = -200f;
    if (kb.IsKeyDown(Microsoft.Xna.Framework.Input.Keys.Right)) vx = 200f;
    Physics.Velocity = new Vector2(vx, Physics.Velocity.Y);

    if (vx == 0) Physics.ApplyFriction(600f, dt);

    // Update physics (applies gravity and moves)
    Physics.Update(dt, ref X, ref Y);

    // Tile collision resolution
    bool hitTop;
    CollisionSystem.ResolveTileCollision(
        ref X, ref Y, 32, 48, _map, out hitTop);
    if (hitTop) Physics.SetGrounded();
    else Physics.IsGrounded = false; // reset each frame
}

// Override Bounds to use actual player size:
public override Rectangle Bounds
    => new Rectangle((int)X, (int)Y, 32, 48);

```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Player falls, lands on tiles, slides with friction – runs and jumps naturally)

■ Warning

Physics.IsGrounded = false before tile collision — tile collision then sets it back to true if the player is standing on something. This pattern ensures grounded is reset each frame.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add coyote time: allow jumping for 0.12 seconds after walking off a ledge.
- 2 Add jump buffering: if Space is pressed 0.1s before landing, fire the jump immediately on contact.
- 3 Add variable jump height: multiply vertical velocity by 0.5f when jump key is released early.
- 4 Add a wall-slide: when pushing against a wall and airborne, cap fall speed at 80f.
- 5 Tune gravity and jump force so the player reaches exactly 3 tile heights (96 pixels).

■ Quiz

Q1. Why multiply velocity by dt?

A. Limits speed

✓ **Frame-rate independent movement — same speed at any FPS**

C. Applies gravity

D. Enables friction

Answer: B

Q2. MaxFallSpeed prevents?

A. Jumping too high

✓ **Tunnelling through thin platforms at high speed**

C. Horizontal drift

D. Gravity accumulating

Answer: B

Q3. Jump() sets Velocity.Y to negative because?

A. Bug

✓ **Screen Y increases downward — negative Y = upward**

C. Convention

D. Required by MonoGame

Answer: B

Q4. ApplyFriction() should be called when?

- A. Always
- B. Only when pressing movement
- ✓ When NOT pressing movement and grounded**
- D. Only airborne

Answer: C

Q5. Physics.IsGrounded must be reset each frame because?

- A. Required
- ✓ Otherwise it stays true after walking off a ledge**
- C. Performance
- D. Thread safety

Answer: B

LESSON

17

Audio — SFX and Music

Sound effects and background music with MonoGame

■ Concept

Game audio divides into two categories. **Sound effects (SFX)**: short clips triggered on demand, can play multiple simultaneously (jump, shoot, coin). **Music**: long tracks, streamed from disk, only one plays at a time, looping continuously.

MonoGame provides **SoundEffect** for SFX (loaded fully into memory, instant playback) and **MediaPlayer** for music (streamed). Add audio files to your Content Pipeline using the MGCB Editor and load them with `Content.Load`.

Add small random **pitch variation** ($\pm 0.1f$) to repeated SFX — without this, every bullet shot or footstep sounds robotic and identical. This one trick dramatically improves audio feel.

■ Project Setup — Add Audio.cs to Engine2D — add .wav files to Content pipeline

■ CREATE FILE

Audio.cs

Static audio manager — call from anywhere in the codebase.

■ CREATE FILE

Content/Content.mgcb

Add your .wav and .ogg files here using the MGCB Editor.

1

Create the Audio manager

`Audio.cs`

Create **Audio.cs**. Use static methods so any class can call `Audio.Play()` without needing a reference.



```
// Audio.cs
using Microsoft.Xna.Framework.Audio;
using Microsoft.Xna.Framework.Media;
using Microsoft.Xna.Framework.Content;
using System.Collections.Generic;

public static class Audio
{
    private static Dictionary<string, SoundEffect> _sfx = new();
    private static float _sfxVolume = 1.0f;
    private static float _musicVolume = 0.6f;

    // Call from Game.LoadContent() to pre-load all sound effects
    public static void LoadSFX(ContentManager content, string name)
        => _sfx[name] = content.Load<SoundEffect>(name);

    // Play a sound effect once – fire and forget
    public static void Play(string name, float pitch = 0f)
    {
        if (!_sfx.TryGetValue(name, out var sfx)) return;
        // Clamp pitch: -1 = one octave down, +1 = one octave up
        sfx.Play(_sfxVolume, Math.Clamp(pitch, -1f, 1f), 0f);
    }

    // Play with random pitch variation – sounds natural
    public static void PlayVaried(string name, float range = 0.1f)
    {
        float pitch = (Random.Shared.NextSingle() * 2 - 1) * range;
        Play(name, pitch);
    }

    // Create a looping instance – for continuous sounds (engine, footsteps)
    public static SoundEffectInstance? CreateLooping(string name)
    {
        if (!_sfx.TryGetValue(name, out var sfx)) return null;
        var inst = sfx.CreateInstance();
        inst.IsLooped = true;
        return inst;
    }

    // Background music
    public static void PlayMusic(Song song, bool repeat = true)
    {
        MediaPlayer.IsRepeating = repeat;
        MediaPlayer.Volume = _musicVolume;
        MediaPlayer.Play(song);
    }
}
```

```
}  
public static void StopMusic() => MediaPlayer.Stop();  
public static void PauseMusic() => MediaPlayer.Pause();  
public static void ResumeMusic()=> MediaPlayer.Resume();  
  
public static float SFXVolume  
{ set => _sfxVolume = Math.Clamp(value, 0, 1); }  
public static float MusicVolume  
{ set { _musicVolume = Math.Clamp(value, 0, 1);  
    MediaPlayer.Volume = _musicVolume; } }  
}
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.

■ Tip

TryGetValue in Play() prevents KeyNotFoundException if a name is mistyped. It silently does nothing instead of crashing — useful in production. During development, you might want to add Console.WriteLine to catch missing names.

2

Load and play sounds from Game1

Game1.cs

In **Game1.cs LoadContent()**, load sounds. Wire jump and coin sounds to input and collision.



Game1.cs

C#

```
// In LoadContent() – load sounds from Content pipeline:
// (Add your .wav files to Content.mgcb first)
// Audio.LoadSFX(Content, "jump");
// Audio.LoadSFX(Content, "coin");
// Audio.LoadSFX(Content, "hurt");

// For testing without real audio files, create silent placeholders:
// (In a real project you would have actual sound files)

// In Update(), after jump input detection:
// if (_input.IsPressed(Keys.Space) && physics.IsGrounded)
// {
//     physics.Jump(420f);
//     Audio.Play("jump"); // normal pitch
// }

// In coin collection:
// Audio.PlayVaried("coin", 0.15f); // ±15% pitch variation

// In TakeDamage:
// Audio.Play("hurt", pitch: -0.05f); // slightly lower pitch

// Volume control:
if (_input.IsHeld(Keys.OemCloseBrackets)) Audio.MusicVolume += 0.01f;
if (_input.IsHeld(Keys.OemOpenBrackets)) Audio.MusicVolume -= 0.01f;
if (_input.IsPressed(Keys.M))
    MediaPlayer.IsMuted = !MediaPlayer.IsMuted;
```

■ Build & Run

\$ dotnet run

Expected output:

(Game runs – audio plays if sound files added to Content pipeline)
M key mutes/unmutes music. [] keys adjust music volume.

■ Tip

Download free game sound effects from freesound.org or kenney.nl. Convert to .wav using Audacity if needed. Add them to your Content.mgcb using the MGCB Editor.

➡ **Exercises — Complete before moving on**

— Exercises — Try These Now

- 1 Add a footstep sound that plays every 0.3 seconds while the player is moving and grounded.
- 2 Create a `SoundEffectInstance` for a wind ambient sound. Start it when on screen, stop it when paused.
- 3 Add master volume control: slider that affects both SFX and music proportionally.
- 4 Make the hurt sound play at a lower pitch as health decreases (map health% to pitch range).
- 5 Implement an `Audio.FadeOutMusic(float duration)` method that reduces volume over time.

■ Quiz

Q1. `SoundEffect` vs `MediaPlayer`?

A. Same

✓ `SoundEffect` = short SFX in memory. `MediaPlayer` = long music streamed from disk

C. `MediaPlayer` is faster

D. `SoundEffect` loops, `MediaPlayer` doesn't

Answer: B

Q2. `PlayVaried` uses random pitch because?

A. Required

✓ Identical pitch every time sounds robotic — variation feels natural

C. More accurate

D. Reduces memory

Answer: B

Q3. `SoundEffectInstance` vs `SoundEffect.Play()`?

A. Same

✓ Instance gives control (pause/stop/loop). `Play()` is fire-and-forget

C. Instance is always faster

D. `Play()` loops automatically

Answer: B

Q4. `MediaPlayer.IsRepeating = true` does?

✓ Plays twice

B. Loops the music continuously

C. Plays backwards

D. Increases volume

Answer: A

Q5. TryGetValue in Play() instead of _sfx[name] because?

- A. Faster
- ✓ Returns false instead of throwing KeyNotFoundException**
- C. Required
- D. Thread-safe

Answer: B

Game State Machine

Managing menu, gameplay, pause, and game-over with clean transitions

■ Concept

A **state machine** controls which mode the game is in. Each state handles its own Update and Draw. When you transition, the current state's OnExit() is called, then the new state's OnEnter(). Without a state machine, Update() becomes a tangle of if/else flags that is impossible to maintain.

Transitions are **queued** — not applied immediately. If you call GoTo() during Update(), the transition happens at the start of the next frame. This prevents half-updated states where some objects use the old state's logic and others use the new.

This is the same architecture as Unity's scene loading (each scene = a state) and Animator Controller (each animation = a state with Enter/Exit transitions).

■ Project Setup — Add state machine files to Engine2D

■ CREATE FILE

IGameState.cs

The state interface all game states must implement.

■ CREATE FILE

StateMachine.cs

The state machine that owns and transitions between states.

■ CREATE FILE

States/MenuState.cs

The main menu state.

■ CREATE FILE

States/PlayingState.cs

The gameplay state.

1

Create IGameState and StateMachine

IGameState.cs

Create **IGameState.cs** and **StateMachine.cs** in the Engine2D folder.

```

IGameState.cs
// IGameState.cs
using Microsoft.Xna.Framework.Graphics;

public interface IGameState
{
    void OnEnter(); // called when becoming active
    void Update(float dt); // called every frame
    void Draw(SpriteBatch sb); // called every frame
    void OnExit(); // called when leaving
}

// StateMachine.cs
public class StateMachine
{
    private IGameState? _current;
    private IGameState? _queued; // pending transition

    // Queue a state change – applied at start of next Update
    public void GoTo(IGameState next) => _queued = next;

    public void Update(float dt)
    {
        // Apply queued transition at start of frame
        if (_queued != null)
        {
            _current?.OnExit();
            _current = _queued;
            _queued = null;
            _current.OnEnter();
        }
        _current?.Update(dt);
    }

    public void Draw(SpriteBatch sb) => _current?.Draw(sb);
}

```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

■ Tip

`_queued != null` before applying the transition ensures `GoTo()` during `Update()` takes effect next frame, not mid-frame. This prevents the scenario where half the objects use the old state.

Create a **States/** folder. Create **MenuState.cs** and **PlayingState.cs** inside it.

```
States/MenuState.cs C#
// States/MenuState.cs
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;

public class MenuState : IGameState
{
    private readonly StateMachine _sm;
    private readonly SpriteBatch _sb;
    private readonly Texture2D _pixel;

    public MenuState(StateMachine sm, SpriteBatch sb, Texture2D px)
    { _sm=sm; _sb=sb; _pixel=px; }

    public void OnEnter() => System.Console.WriteLine("Menu entered");
    public void OnExit() => System.Console.WriteLine("Menu exited");

    public void Update(float dt)
    {
        if (Keyboard.GetState().IsKeyDown(Keys.Enter))
            _sm.GoTo(new PlayingState(_sm, _sb, _pixel));
    }

    public void Draw(SpriteBatch sb)
    {
        sb.Begin();
        // Draw menu background and text (text rendering in next lesson)
        sb.Draw(_pixel, new Rectangle(0,0,800,600), Color.DarkBlue * 0.8f);
        sb.End();
    }
}

// States/PlayingState.cs - simplified
public class PlayingState : IGameState
{
    private readonly StateMachine _sm;
    private readonly SpriteBatch _sb;
    private readonly Texture2D _pixel;
    private Scene _scene = new Scene();

    public PlayingState(StateMachine sm, SpriteBatch sb, Texture2D px)
    { _sm=sm; _sb=sb; _pixel=px; }

    public void OnEnter()
    {
        System.Console.WriteLine("Game started");
    }
}
```

```

States/MenuState.cs (cont.)
_scene = new Scene(); // fresh scene each time
}
public void OnExit() => System.Console.WriteLine("Game ended");

public void Update(float dt) => _scene.Update(dt);

public void Draw(SpriteBatch sb)
{
    sb.Begin();
    sb.Draw(_pixel, new Rectangle(0,0,800,600), Color.DarkSlateBlue);
    _scene.Draw(sb);
    sb.End();
}
}

```

■ Build & Run

```
$ dotnet build
```

Expected output:

Build succeeded.

3

Wire the state machine into Game1

Game1.cs

Replace the **Game1** update and draw logic to delegate everything to the state machine.

```

Game1.cs
// Add field:
private StateMachine _sm = new StateMachine();

// In LoadContent(), after _pixel creation:
_sm.GoTo(new MenuState(_sm, _spriteBatch, _pixel));

// Replace Update() body with:
if (Keyboard.GetState().IsKeyDown(Keys.Escape)) Exit();
float dt = (float)gameTime.ElapsedGameTime.TotalSeconds;
dt = Math.Min(dt, 0.05f);
_sm.Update(dt);
base.Update(gameTime);

// Replace Draw() body with:
GraphicsDevice.Clear(Color.Black);
_sm.Draw(_spriteBatch);
base.Draw(gameTime);

```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
Menu entered
(Dark blue screen – press Enter)
Menu exited
Game started
(Dark slate blue screen – game running)
```

■ Tip

Every future game mode — pause, game-over, level select — is just a new `IGameState` implementation. The `StateMachine` does not need to change at all.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Add a `PausedState`: pressing `Escape` during `PlayingState` pushes a `PausedState`. Pressing `Escape` again returns.
- 2 Add a `GameOverState` that displays a 'GAME OVER' rectangle and returns to `MenuState` on `Enter`.
- 3 Add a transition timer: states take 0.5 seconds to fade out before the new state fades in.
- 4 Implement `sm.Push()` / `sm.Pop()` for a state stack — push paused on top of playing, pop to resume.
- 5 Make `PlayingState` save and restore scene state when pushing/popping pause (objects keep their positions).

■ Quiz

Q1. Why queue transitions instead of applying immediately?

A. Performance

✓ Prevents mid-frame corruption where some objects use old state, others use new

C. Required by `MonoGame`

D. Thread safety

Answer: B

Q2. What does `OnEnter()` do?

A. Runs at game launch

✓ Called when this state becomes active — initialise its resources

C. Every frame

D. Random

Answer: B

Q3. What does OnExit() do?

A. Crash handler

✓ Called when leaving this state — clean up its resources

C. Every frame

D. Called by base

Answer: B

Q4. Unity equivalent of state machine?

A. MonoBehaviour

B. Rigidbody

✓ SceneManager.LoadScene() + Animator Controller

D. Camera

Answer: C

Q5. State stack (Push/Pop) vs simple GoTo?

A. Same

✓ Stack allows returning to previous state — GoTo does not remember where you came from

C. Stack is slower

D. GoTo is safer

Answer: B

PART 3

3D ENGINE SYSTEMS

Extend every 2D concept into three dimensions

LESSON

19

Vector3 and 3D Space

Adding a Z axis — position, direction, and distance

■ Concept

Moving to 3D adds a Z axis. Your game loop, scene manager, input, and audio stay the same. The math changes. Three operations are fundamental: **Dot product** (cosine of angle between vectors: 1=same, 0=perpendicular, -1=opposite). **Cross product** (vector perpendicular to both — used to find camera Right). **Normalize** (set length to 1, pure direction).

Always normalize direction vectors before multiplying by speed. (1,1,0) has length $\sqrt{2} \approx 1.41$ without normalization — diagonal movement becomes 41% faster.

■ Project Setup — Create the Engine3D project



Terminal

```
dotnet new console -n Engine3D
cd Engine3D
dotnet add package MonoGame.Framework.DesktopGL
```

■ CREATE FILE

Program.cs

Replace to experiment with Vector3 math.

1

Explore Vector3 operations

Program.cs

Replace **Program.cs** with this console test.

```

Program.cs
using Microsoft.Xna.Framework;
Vector3 a=new(1,0,0); Vector3 b=new(0,1,0);

float dot = Vector3.Dot(a,b);
Console.WriteLine($"Dot(right,up) = {dot}"); // 0 – perpendicular

Vector3 cross = Vector3.Cross(a,b);
Console.WriteLine($"Cross = {cross}"); // (0,0,-1)

Vector3 norm = Vector3.Normalize(new Vector3(2,2,2));
Console.WriteLine($"Normalize len = {norm.Length():F4}"); // 1.0000

float distSq = Vector3.DistanceSquared(Vector3.Zero, new(3,4,0));
Console.WriteLine($"DistSq = {distSq}"); // 25 – no sqrt needed

// 3D gravity simulation
Vector3 pos=new(0,10,0); Vector3 vel=Vector3.Zero;
Vector3 grav=new(0,-9.8f,0); float dt=0.1f;
for(int i=0;i<5;i++){
    vel+=grav*dt; pos+=vel*dt;
    Console.WriteLine($"Y={pos.Y:F2} vy={vel.Y:F2}");
}

```

■ Build & Run

\$ dotnet run

Expected output:

```

Dot(right,up) = 0
Cross = {X:0 Y:0 Z:-1}
Normalize len = 1.0000
DistSq = 25
Y=9.90 vy=-0.98
Y=9.61 vy=-1.96
Y=9.12 vy=-2.94
Y=8.43 vy=-3.92
Y=7.54 vy=-4.90

```

■ Tip

DistanceSquared avoids Math.Sqrt. For 'is A closer than B' comparisons, compare distSq values — same answer, no sqrt cost.

2

Camera direction vectors

Program.cs

Add below the previous code to practice forward/right math.



Program.cs

C#

```
float yaw=MathHelper.ToRadians(45f);
Vector3 fwd=Vector3.Normalize(new(MathF.Sin(yaw),0,-MathF.Cos(yaw)));
Vector3 rgt=Vector3.Normalize(Vector3.Cross(fwd,Vector3.Up));
Console.WriteLine($"Fwd: {fwd}");
Console.WriteLine($"Rgt: {rgt}");
Console.WriteLine($"Fwd·Rgt = {Vector3.Dot(fwd,rgt):F5}"); // 0.00000
```

■ Build & Run

\$ dotnet run

Expected output:

```
Fwd: {X:0.707 Y:0 Z:-0.707}
Rgt: {X:0.707 Y:0 Z:0.707}
Fwd·Rgt = 0.00000
```

■ Tip

Cross(Forward, Up) always gives the Right vector. This works regardless of what direction Forward is pointing.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Compute Dot((1,0,0),(-1,0,0)). Interpret the result in plain English.
- 2 Normalize 5 different arbitrary Vector3 values and verify each has length 1.0000.
- 3 Simulate a ball thrown at 45° with speed 10m/s. Print Y position every 0.1s for 2 seconds.
- 4 Write FaceToward(Vector3 from, Vector3 to) returning the normalised direction.
- 5 Find the point 1/3 of the way from A=(0,0,0) to B=(9,6,3) using Lerp.

■ Quiz

Q1. Dot product = 0 means?

A. Same direction

✓ Perpendicular

C. Opposite

D. Equal lengths

Answer: B

Q2. Why DistanceSquared?

- A. More accurate
- ✓ Avoids sqrt — equivalent result**
- C. Required
- D. Handles negatives

Answer: B

Q3. Cross(Forward, Up) gives?

- A. Down
- ✓ Right — perpendicular to Forward**
- D. Forward again

Answer: C

Q4. Normalize sets length to?

- A. 0
- B. Infinity
- ✓ 1**
- D. Original value

Answer: C

Q5. Vector3.Lerp(a,b,0.25f)?

- A. a
- B. b
- ✓ 25% from a to b**
- D. Midpoint

Answer: C

LESSON

20

The Three Matrices

World, View, Projection — the GPU pipeline

■ Concept

Three matrix transformations turn 3D geometry into 2D pixels. **World**: positions and orients the object (Scale × Rotate × Translate). **View**: camera position and orientation. **Projection**: 3D→2D with perspective (far=small). MonoGame's BasicEffect accepts all three.

■ Project Setup — Continue in Engine3D — update Game1.cs

■ CREATE FILE

Game1.cs

Replace to render a spinning triangle.

1

Render a triangle with three matrices

Game1.cs

Replace Game1.cs.



```
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
public class Game1:Game{
    GraphicsDeviceManager _gfx; BasicEffect _fx;
    VertexBuffer _vb; float _rot=0f;
    public Game1(){_gfx=new GraphicsDeviceManager(this);
        _gfx.PreferredBackBufferWidth=800;
        _gfx.PreferredBackBufferHeight=600;
        Content.RootDirectory="Content";}
    protected override void LoadContent(){
        _fx=new BasicEffect(GraphicsDevice);
        _fx.VertexColorEnabled=true;
        var v=new VertexPositionColor[]{
            new(new Vector3(0,1,0),Color.Red),
            new(new Vector3(-1,-1,0),Color.Green),
            new(new Vector3(1,-1,0),Color.Blue)};
        _vb=new VertexBuffer(GraphicsDevice,
            VertexPositionColor.VertexDeclaration,3,BufferUsage.WriteOnly);
        _vb.SetData(v);}
    protected override void Update(GameTime gt){
        if(Keyboard.GetState().IsKeyDown(Keys.Escape))Exit();
        _rot+=(float)gt.ElapsedGameTime.TotalSeconds*1.5f;
        base.Update(gt);}
    protected override void Draw(GameTime gt){
        GraphicsDevice.Clear(Color.Black);
        GraphicsDevice.SetVertexBuffer(_vb);
        _fx.World=Matrix.CreateRotationY(_rot);
        _fx.View=Matrix.CreateLookAt(new Vector3(0,0,5),Vector3.Zero,Vector3.Up);
        _fx.Projection=Matrix.CreatePerspectiveFieldOfView(
            MathHelper.ToRadians(60f),
            GraphicsDevice.Viewport.AspectRatio,0.1f,1000f);
        foreach(var p in _fx.CurrentTechnique.Passes){
            p.Apply();
            GraphicsDevice.DrawPrimitives(PrimitiveType.TriangleList,0,1);}
        base.Draw(gt);}}
```

■ Build & Run

\$ dotnet run

Expected output:

(Coloured triangle spins in 3D. Press Escape to close.)

■ Tip

SRT order: Scale first, Rotate second, Translate last. Other orders produce unexpected results.

Add these fields and Update code.

```
Game1.cs C#
Vector3 _cam=new(0,0,5);
// In Update(), add:
float dt=(float)gt.ElapsedGameTime.TotalSeconds;
var kb=Keyboard.GetState();
float s=3f;
if(kb.IsKeyDown(Keys.W))_cam.Z-=s*dt;
if(kb.IsKeyDown(Keys.S))_cam.Z+=s*dt;
if(kb.IsKeyDown(Keys.A))_cam.X-=s*dt;
if(kb.IsKeyDown(Keys.D))_cam.X+=s*dt;
Window.Title=$"Camera:{_cam}";
// In Draw(), update View:
_fx.View=Matrix.CreateLookAt(_cam,Vector3.Zero,Vector3.Up);
```

■ Build & Run

\$ dotnet run

Expected output:

(WASD moves camera. Title shows position.)

■ Tip

Try pressing W past the origin — you pass through the triangle and it appears mirrored. The camera is now in front of it.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Place a second triangle at (2,0,0) by changing World before a second DrawPrimitives call.
- 2 Add Scale: `Matrix.CreateScale(2f)*Matrix.CreateRotationY(_rot)`.
- 3 Circle the camera: `_cam=new(MathF.Sin(t)*5,0,MathF.Cos(t)*5)` where t increments with dt.
- 4 Set `nearPlane=3f` — objects within 3 units disappear. Observe.
- 5 Try FOV 30 vs 100. Describe the visual difference.

■ Quiz

Q1. World matrix positions?

- A. Camera
- B. Applies perspective
- ✓ The object**
- D. Clears depth

Answer: C

Q2. CreateLookAt creates?

- A. World
- ✓ View — camera orientation**
- C. Projection
- D. Rotation only

Answer: B

Q3. Smaller FOV?

- A. Wider view
- ✓ Telephoto — objects look larger**
- C. No change
- D. Inverted

Answer: B

Q4. SRT order?

- A. Texture/Render/Shader
- ✓ Scale then Rotate then Translate**
- C. Source/Result/Target
- D. Sort/Remove/Transform

Answer: B

Q5. Why iterate Passes?

- A. Style
- ✓ Some effects need multiple rendering passes**
- C. Required
- D. Performance

Answer: B

LESSON

21

Camera3D — Perspective and Look-At

Mouse-look FPS camera

■ Concept

Camera defined by position, yaw/pitch-derived forward, and FOV. Mouse X→yaw, mouse Y→pitch. WASD uses flat forward (Y=0) to stay grounded. Lock mouse to screen centre by calling `Mouse.SetPosition` every frame — $\text{delta} = \text{current} - \text{centre}$.

■ Project Setup — Add Camera3D.cs to Engine3D

■ CREATE FILE

Camera3D.cs

Free-look 3D camera.

■ CREATE FILE

Game1.cs

Wire camera.

1

Create Camera3D.cs

Camera3D.cs

Create **Camera3D.cs**.



Camera3D.cs

C#

```
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class Camera3D{
    public Vector3 Position=new(0,2,5);
    public float Yaw=0,Pitch=0,Fov=60f;
    const float S=0.003f; int _cx,_cy;
    public void Init(int w,int h){_cx=w/2;_cy=h/2;Mouse.SetPosition(_cx,_cy);}
    public void ProcessMouseDelta(float dx,float dy)
    { Yaw+=dx*S;
      Pitch=MathHelper.Clamp(Pitch-dy*S,
        -MathHelper.PiOver2+0.01f,MathHelper.PiOver2-0.01f);}
    public void ProcessMouse()
    { var ms=Mouse.GetState();
      ProcessMouseDelta(ms.X-_cx,ms.Y-_cy);
      Mouse.SetPosition(_cx,_cy);}
    public Vector3 Forward=>Vector3.Normalize(new(
      MathF.Cos(Pitch)*MathF.Sin(Yaw),MathF.Sin(Pitch),
      MathF.Cos(Pitch)*MathF.Cos(Yaw)));
    public Vector3 Right=>Vector3.Normalize(Vector3.Cross(Forward,Vector3.Up));
    public Matrix GetView()=>Matrix.CreateLookAt(Position,Position+Forward,Vector3.Up);
    public Matrix GetProjection(float asp)=>
      Matrix.CreatePerspectiveFieldOfView(MathHelper.ToRadians(Fov),asp,0.1f,1000f);
    public void MoveWASD(KeyboardState kb,float spd,float dt){
      var f=Vector3.Normalize(new(Forward.X,0,Forward.Z));
      var r=Right;
      if(kb.IsKeyDown(Keys.W))Position+=f*spd*dt;
      if(kb.IsKeyDown(Keys.S))Position-=f*spd*dt;
      if(kb.IsKeyDown(Keys.A))Position-=r*spd*dt;
      if(kb.IsKeyDown(Keys.D))Position+=r*spd*dt;
      if(kb.IsKeyDown(Keys.Q))Position+=Vector3.Up*spd*dt;
      if(kb.IsKeyDown(Keys.E))Position-=Vector3.Up*spd*dt;}}
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Warning

Pitch clamped to $\pm 89.4^\circ$. At exactly $\pm 90^\circ$ Forward is parallel to Up — LookAt breaks and the camera flips violently.

2

Wire Camera3D into Game1

Game1.cs

Add field and update Draw/Update.



Game1.cs

C#

```
Camera3D _camera=new Camera3D();
// In LoadContent(): _camera.Init(800,600);
// In Update():
float dt=(float)gt.ElapsedGameTime.TotalSeconds;
_camera.ProcessMouse();
_camera.MoveWASD(Keyboard.GetState(),5f,dt);
// In Draw(), replace View/Projection:
_fx.View=_camera.GetView();
_fx.Projection=_camera.GetProjection(GraphicsDevice.Viewport.AspectRatio);
```

■ Build & Run

\$ dotnet run

Expected output:

(Mouse looks around. WASD flies. Triangle stays in world.)

■ Tip

Move behind the triangle with S then turn around with the mouse — you see the back face. This is the foundation for all 3D navigation.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add Shift sprint (3× speed).
- 2 Add FOV breathing: sprint reduces to 50, release restores to 60 with MathHelper.Lerp.
- 3 Press Tab to toggle mouse capture — free cursor when not captured.
- 4 Print Forward each frame — verify length always 1.0000.
- 5 Implement orbit around origin: camera circles at radius 5 using sin/cos on position.

■ Quiz

Q1. Pitch clamp $\pm 89^\circ$ prevents?

A. Looking back

✓ LookAt breaking — violent flip

C. Shake

D. 2D gimbal lock

Answer: B

Q2. Flat forward (Y=0)?

A. Slow

✓ Prevents flying

C. Required

D. Style

Answer: B

Q3. Right=Cross(Forward,Up)?

A. Down

B. Up

✓ Perpendicular right vector

D. Forward again

Answer: C

Q4. Re-centre mouse prevents?

A. Lag

✓ Cursor hitting edge and rotation stopping

C. Leak

D. Required

Answer: B

Q5. Pitch affects Forward.Y?

A. Not at all

✓ $\text{Forward.Y} = \text{MathF.Sin}(\text{Pitch})$

C. Direct copy

D. $\cos(\text{Pitch})$

Answer: B

LESSON

22

Quaternions and 3D Rotation

Smooth 3D rotation without gimbal lock

■ Concept

Gimbal lock: when two Euler axes align at 90° you lose a degree of freedom. **Quaternions** avoid this. Use factory methods: `CreateFromAxisAngle`, `Slerp`, `Concatenate`. Always convert to `Matrix` with `CreateFromQuaternion` before passing to `BasicEffect`.

■ Project Setup — Extend Engine3D — update Game1.cs

■ CREATE FILE

Game1.cs

Add quaternion rotation.

1

Quaternion-controlled rotation

Game1.cs

Add these fields and Update code.

```
Game1.cs C#
Quaternion _qRot=Quaternion.Identity;
Quaternion _qTarget=Quaternion.Identity;
float _st=0f;
// In Update():
var kb2=Keyboard.GetState();
if(kb2.IsKeyDown(Keys.D1)) _qTarget=Quaternion.Identity;
if(kb2.IsKeyDown(Keys.D2)) _qTarget=
    Quaternion.CreateFromAxisAngle(Vector3.Up,MathHelper.ToRadians(90f));
if(kb2.IsKeyDown(Keys.D3)) _qTarget=
    Quaternion.CreateFromAxisAngle(Vector3.Right,MathHelper.ToRadians(45f));
if(kb2.IsKeyDown(Keys.R)) _st=0f;
_st=Math.Min(_st+dt*0.8f,1f);
_qRot=Quaternion.Slerp(_qRot,_qTarget,_st);
// In Draw() replace World:
_fx.World=Matrix.CreateFromQuaternion(_qRot);
Window.Title=$"1=identity 2=90°Y 3=45°X R=reset t={_st:F2}";
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Keys 1/2/3 set target rotation. Object smoothly interpolates to it.)

■ Tip

Slerp interpolates along the shortest arc. Press R then another key to see the interpolation from the start.

2

Combine quaternions

Program.cs

Add to a temp console test to understand Concatenate order.

```
Program.cs C#
var spinY=Quaternion.CreateFromAxisAngle(Vector3.Up,MathHelper.ToRadians(90f));
var tiltX=Quaternion.CreateFromAxisAngle(Vector3.Right,MathHelper.ToRadians(30f));
var combined=Quaternion.Concatenate(tiltX,spinY); // tiltX applied first
var m=Matrix.CreateFromQuaternion(combined);
Console.WriteLine($"Right: {m.Right}");
var fromEuler=Quaternion.CreateFromYawPitchRoll(MathHelper.ToRadians(90f),0,0);
Console.WriteLine($"90°Y: {fromEuler}");
```

■ Build & Run

\$ dotnet run

Expected output:

Right: ...

90°Y: {X:0 Y:0.707 Z:0 W:0.707}

■ Warning

Concatenate(A,B) applies A first then B. Comment order clearly in your code.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Tumbling cube: rotate on all three axes simultaneously using Concatenate.
- 2 Smooth look-at: Slerp current rotation to face a target position.
- 3 Every 2 seconds, Slerp to a new random 180° rotation.
- 4 Compare Lerp vs Slerp on quaternions — observe speed difference mid-arc.
- 5 Verify: rotate X=90° then Y with Euler angles — observe gimbal lock.

■ Quiz

Q1. Gimbal lock?

✓ **Physics**

- B. Two Euler axes align at 90° — degree of freedom lost
- C. Rendering artefact
- D. Float precision

Answer: A

Q2. Slerp vs Lerp?

A. Same

✓ **Slerp=constant speed sphere arc. Lerp=uneven speed**

- C. Lerp more accurate
- D. No difference

Answer: B

Q3. CreateFromQuaternion?

A. Matrix to quat

✓ **Quat to rotation Matrix**

- C. Euler to matrix
- D. Quat to Vector3

Answer: B

Q4. Concatenate(A,B) applies?

A. B then A

✓ **A then B**

- C. Simultaneously
- D. Random

Answer: B

Q5. Quaternion.Identity?

A. Random

✓ **No rotation**

- C. 180°
- D. Maximum

Answer: B

LESSON

23

3D Mesh and Vertex Buffers

Upload geometry once, draw every frame

■ Concept

A **VertexBuffer** holds vertex data on the GPU. An **IndexBuffer** holds indices so vertices can be shared. Upload with `SetData()` ONCE — it's slow. Draw with `DrawIndexedPrimitives()` every frame — it's fast. A cube needs 24 vertices (4 per face, different normals per face) and 36 indices.

■ Project Setup — Create Mesh.cs in Engine3D

■ CREATE FILE

Mesh.cs

Wraps VB+IB with Draw method.

■ CREATE FILE

Game1.cs

Render a quad.

1

Create Mesh.cs

Mesh.cs

Create **Mesh.cs**.

```
Mesh.cs C#
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Graphics;
public class Mesh{
    VertexBuffer _vb; IndexBuffer _ib;
    int _tris; GraphicsDevice _gd;
    public Mesh(GraphicsDevice gd,
        VertexPositionNormalTexture[] verts, short[] idx){
        _gd=gd; _tris=idx.Length/3;
        _vb=new VertexBuffer(gd,
            VertexPositionNormalTexture.VertexDeclaration,
            verts.Length,BufferUsage.WriteOnly);
        _vb.SetData(verts);
        _ib=new IndexBuffer(gd,IndexElementSize.SixteenBits,
            idx.Length,BufferUsage.WriteOnly);
        _ib.SetData(idx);}
    public void Draw(BasicEffect fx){
        _gd.SetVertexBuffer(_vb); _gd.Indices=_ib;
        foreach(var p in fx.CurrentTechnique.Passes){
            p.Apply();
            _gd.DrawIndexedPrimitives(PrimitiveType.TriangleList,0,0,_tris);}}}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.  
0 Warning(s)
```

■ Tip

SetData is slow — run it once in LoadContent. DrawIndexedPrimitives is fast — data stays on GPU.

2

Build and draw a front face quad

Game1.cs

Add to LoadContent and Draw.

```
Game1.cs C#  
Mesh _quad; // add field  
// In LoadContent():  
var v=new VertexPositionNormalTexture[4];  
v[0]=new(new Vector3(-1,-1,0),Vector3.Backward,new Vector2(0,1));  
v[1]=new(new Vector3( 1,-1,0),Vector3.Backward,new Vector2(1,1));  
v[2]=new(new Vector3( 1, 1,0),Vector3.Backward,new Vector2(1,0));  
v[3]=new(new Vector3(-1, 1,0),Vector3.Backward,new Vector2(0,0));  
short[] idx={0,1,2, 0,2,3};  
_quad=new Mesh(GraphicsDevice,v,idx);  
_fx.EnableDefaultLighting();  
_fx.DiffuseColor=Color.SteelBlue.ToVector3();  
// In Draw(), replace DrawPrimitives:  
_fx.World=Matrix.CreateFromQuaternion(_qRot);  
_quad.Draw(_fx);
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
(Blue lit quad rotates with quaternion keys 1/2/3.)
```

■ Tip

Normal=Vector3.Backward makes this face lit when the light is in front. Without normals, EnableDefaultLighting has nothing to compute — the face appears uniform grey.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add 5 more faces to make a complete cube (6 faces, 24 verts, 36 indices).
- 2 Load a PNG via `Content.Load<Texture2D>`, set `_fx.Texture` and `TextureEnabled=true`.
- 3 Create a flat Ground mesh (2 triangles, Y=0 plane).
- 4 Create a Pyramid (4 triangular faces + square base).
- 5 Create MeshRenderer class: holds Mesh and WorldMatrix, exposes `Draw(Camera3D, BasicEffect)`.

■ Quiz

Q1. Upload once because?

- A. Required
- ✓ **SetData slow — GPU data persists across frames**
- C. Memory
- D. Required by BasicEffect

Answer: B

Q2. 24 verts for cube not 8?

- ✓ **Required**
- B. Each face needs unique normals — corners can't be shared across faces
- C. Texture
- D. Performance

Answer: A

Q3. DrawIndexedPrimitives vs DrawPrimitives?

- A. Same
- ✓ **Indexed reuses vertices. Primitives duplicates all verts**
- C. Indexed slower
- D. No normals

Answer: B

Q4. VertexPositionNormalTexture has?

- A. Position only
- B. Pos+colour
- ✓ **Pos+normal+UV**
- D. Pos+index

Answer: C

Q5. BufferUsage.WriteOnly hints?

A. Dynamic

✓ **Write-once read-many — GPU can optimise**

C. Required

D. Stream

Answer: B

LESSON

24

Lighting — The Phong Model

Ambient, diffuse, specular with BasicEffect

■ Concept

Without lighting every face is equally bright — looks flat. **Phong model**: **Ambient** (constant base so shadows aren't black), **Diffuse** (brightness $\propto$ angle to light — key shading), **Specular** (shiny highlight from reflected light + view angle). `BasicEffect.EnableDefaultLighting()` applies a 3-light Phong setup automatically.

■ Project Setup — Continue in Engine3D — configure lighting

■ CREATE FILE

Game1.cs

Set up and interact with lighting.

1

Configure BasicEffect lighting

Game1.cs

Replace `_fx` setup in `LoadContent`.

```
Game1.cs C#
_fx=new BasicEffect(GraphicsDevice);
_fx.EnableDefaultLighting();
_fx.DirectionalLight0.Direction=
    Vector3.Normalize(new Vector3(-1,-1,-0.5f));
_fx.DirectionalLight0.DiffuseColor=Color.White.ToVector3();
_fx.DirectionalLight0.SpecularColor=Color.White.ToVector3()*0.5f;
_fx.DirectionalLight1.Enabled=true;
_fx.DirectionalLight1.Direction=Vector3.Normalize(new Vector3(1,-0.3f,0));
_fx.DirectionalLight1.DiffuseColor=new Vector3(0.3f,0.3f,0.4f);
_fx.AmbientLightColor=new Vector3(0.1f,0.1f,0.15f);
_fx.DiffuseColor=Color.SteelBlue.ToVector3();
_fx.SpecularColor=Color.White.ToVector3();
_fx.SpecularPower=64f;
```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Cube shows Phong lighting — bright face, dark face, shiny highlight)

■ Tip

SpecularPower: 8=matte, 64=polished, 256=metallic, 512=mirror. Same as Unity 'Smoothness'.

Add to Update.

```
Game1.cs C#
float _la=0f; // field
// In Update():
var kb3=Keyboard.GetState();
if(kb3.IsKeyDown(Keys.Left)) _la-=dt;
if(kb3.IsKeyDown(Keys.Right)) _la+=dt;
_fx.DirectionalLight0.Direction=Vector3.Normalize(
    new Vector3(MathF.Sin(_la), -0.8f, MathF.Cos(_la)));
_fx.AmbientLightColor=kb3.IsKeyDown(Keys.A)
    ? Vector3.Zero : new Vector3(0.1f, 0.1f, 0.15f);
if(kb3.IsKeyDown(Keys.OemPlus))
    _fx.SpecularPower=Math.Min(_fx.SpecularPower+20*dt, 512);
if(kb3.IsKeyDown(Keys.OemMinus))
    _fx.SpecularPower=Math.Max(_fx.SpecularPower-20*dt, 1);
Window.Title=$"Shininess:{_fx.SpecularPower:F0} A=no ambient";
```

■ Build & Run

\$ dotnet run

Expected output:

L/R arrows rotate light. A removes ambient. +/- changes shininess.

■ Tip

Hold A — shadow faces go black. Release — ambient fills them. This is the clearest demonstration of what ambient does.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add a rim light from positive Z (behind the object).
- 2 Change DiffuseColor to Color.Gold.ToVector3().
- 3 Set SpecularColor=Color.Red.ToVector3() — red metallic highlights.
- 4 Zero both DiffuseColor values — only ambient remains. Observe.
- 5 Rotate the primary light in a full circle over 10 seconds using elapsed time.

■ Quiz

Q1. Diffuse depends on?

A. View direction

✓ **Angle normal↔light**

C. Distance

D. Texture

Answer: B

Q2. Ambient prevents?

A. Specular

✓ **Shadow faces being pitch-black**

C. Diffuse

D. Texture mapping

Answer: B

Q3. Higher SpecularPower?

A. Larger highlight

✓ **Smaller sharper highlight**

C. Brighter ambient

D. Stronger diffuse

Answer: B

Q4. EnableDefaultLighting sets up?

A. One point light

✓ **Three directional Phong lights**

C. Spot lights

D. No lights

Answer: B

Q5. Specular depends on?

A. Normal only

✓ **Reflected light and view angle — varies with camera**

C. Texture

D. Ambient

Answer: B

LESSON

25

3D Physics and Sphere Collision

Velocity, gravity, and collision with Vector3

■ Concept

3D physics extends 2D PhysicsBody: replace Vector2 with Vector3, gravity=Vector3(0,-9.8f,0). Sphere collision: two spheres overlap when $\text{distanceSquared}(A,B) \leq (rA+rB)^2$. Push vector: $\text{normalise}(A.\text{centre}-B.\text{centre}) \times \text{overlap}$.

■ Project Setup — Add PhysicsBody3D.cs and Sphere3D.cs to Engine3D

■ CREATE FILE

PhysicsBody3D.cs

3D physics.

■ CREATE FILE

Sphere3D.cs

Sphere collision.

■ CREATE FILE

Game1.cs

Falling spheres test.

1

Create PhysicsBody3D and Sphere3D

PhysicsBody3D.cs

Create both files.

```

PhysicsBody3D.cs
// PhysicsBody3D.cs
using Microsoft.Xna.Framework;
public class PhysicsBody3D{
    public Vector3 Velocity{get;set;}=Vector3.Zero;
    public float Mass{get;set;}=1f;
    static readonly Vector3 G=new(0,-9.8f,0);
    public void Update(float dt,ref Vector3 pos){
        Velocity+=G*dt;
        if(Velocity.Y<-50f)Velocity=new(Velocity.X,-50f,Velocity.Z);
        pos+=Velocity*dt;}
    public void ApplyImpulse(Vector3 i)=>Velocity+=i;
    public void ApplyForce(Vector3 f)=>Velocity+=f/Mass;}

// Sphere3D.cs
using Microsoft.Xna.Framework;
public struct Sphere3D{
    public Vector3 C; public float R;
    public Sphere3D(Vector3 c,float r){C=c;R=r;}
    public bool Overlaps(Sphere3D o)
    { float r=R+o.R; return Vector3.DistanceSquared(C,o.C)<=r*r; }
    public Vector3 Push(Sphere3D o)
    { float ov=(R+o.R)-Vector3.Distance(C,o.C);
      return Vector3.Normalize(C-o.C)*ov; }}

```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

2

Falling spheres test

Game1.cs

Add fields and simulation code.



Game1.cs

C#

```
Vector3[] _sp=new Vector3[5];
PhysicsBody3D[] _ph=new PhysicsBody3D[5];
// In LoadContent():
var rng=new System.Random(42);
for(int i=0;i<5;i++){
    _sp[i]=new((float)(rng.NextDouble()*4-2),
               (float)(rng.NextDouble()*8+2),
               (float)(rng.NextDouble()*4-2));
    _ph[i]=new PhysicsBody3D();}
// In Update(), after camera:
for(int i=0;i<5;i++){
    _ph[i].Update(dt,ref _sp[i]);
    if(_sp[i].Y<-3f){ // ground at Y=-3
        _sp[i].Y=-3f;
        var v=_ph[i].Velocity;
        _ph[i].Velocity=new(v.X, -v.Y*0.35f,v.Z);}
}
Window.Title=$"Y={_sp[0].Y:F2} vy={_ph[0].Velocity.Y:F2}";
```

■ Build & Run

\$ dotnet run

Expected output:

(5 spheres fall, bounce at Y=-3, settle. Title shows sphere 0 state.)

■ Tip

Bounce coefficient 0.35 = 35% energy retained. 0=dead stop. 1=infinite bounce. Real materials are 0.3-0.6.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Sphere-sphere bounce: when two overlap, push both apart and reflect velocities.
- 2 Add horizontal drag: reduce X/Z velocity by 0.98 each frame when grounded.
- 3 Use MonoGame's BoundingBox for 3D AABB — test BoundingBox.Intersects(BoundingSphere).
- 4 Add attractor: one sphere applies ApplyForce toward it to all others.
- 5 Player input: WASD applies force to sphere 0 horizontally.

■ Quiz

Q1. DistanceSquared for sphere overlap?

A. Accurate

✓ Avoids sqrt — same result

C. Required

D. Handles negatives

Answer: B

Q2. ApplyImpulse vs ApplyForce?

A. Same

✓ Impulse=instant. Force=gradual/mass

C. Force instant

D. Impulse ignores mass

Answer: B

Q3. Bounce coefficient 0.35?

A. Perfect

✓ 35% energy retained — realistic damping

C. No bounce

D. Increase each hit

Answer: B

Q4. 3D AABB extends 2D by?

A. Circles

✓ Adding Z axis overlap

C. Quaternions

D. Slower

Answer: B

Q5. Push = normalize(A-B)*overlap?

A. Random

✓ Direction A from B, scaled by depth — separates them

C. Attract

D. Half

Answer: B

LESSON

26

Model Loading and Scene3D

Loading .fbx models and the 3D scene

■ Concept

MonoGame imports FBX/OBJ via the MGCB Editor. A loaded **Model** contains **ModelMesh** objects — set **World/View/Projection** on each mesh's **BasicEffect** then call **mesh.Draw()**. **Scene3D** mirrors **Scene2D** exactly — same pending-add buffer, same cleanup loop, **Draw()** takes **Camera3D**.

■ Project Setup — Add Scene3D.cs to Engine3D

■ CREATE FILE

GameObject3D.cs

3D base class.

■ CREATE FILE

Scene3D.cs

3D scene manager.

■ CREATE FILE

Content/Content.mgcb

Add .fbx via MGCB Editor.

1

Create GameObject3D and Scene3D

Scene3D.cs

Create both files.


```

Scene3D.cs
C#
// GameObject3D.cs
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Graphics;
public class GameObject3D{
    public Vector3 Position{get;set;}=Vector3.Zero;
    public Quaternion Rotation{get;set;}=Quaternion.Identity;
    public Vector3 Scale{get;set;}=Vector3.One;
    public bool IsActive{get;set;}=true;
    public bool IsDestroyed{get;set;}
    public Matrix World=>Matrix.CreateScale(Scale)
        *Matrix.CreateFromQuaternion(Rotation)
        *Matrix.CreateTranslation(Position);
    public virtual void Update(float dt){}
    public virtual void Draw(Camera3D cam,GraphicsDevice gd){}}

// Scene3D.cs – identical loop to Scene2D
using System.Collections.Generic; using System.Linq;
public class Scene3D{
    List<GameObject3D> _o=new(),_q=new();
    public void Add(GameObject3D o)=>_q.Add(o);
    public void Update(float dt){
        if(_q.Count>0){_o.AddRange(_q);_q.Clear();}
        foreach(var o in _o)if(o.IsActive)o.Update(dt);}
    public void Draw(Camera3D c,GraphicsDevice gd)
        {foreach(var o in _o)if(o.IsActive)o.Draw(c,gd);}
    public void Cleanup()=>_o.RemoveAll(o=>o.IsDestroyed);
    public System.Collections.Generic.IEnumerable<T> GetAll<T>() where T:GameObject3D
        =>_o.OfType<T>();}

```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
    0 Warning(s)
```

■ Tip

Scene3D.Update and Draw are word-for-word the same as Scene2D. Only the type name and Draw() signature differ.

2

ModelObject3D — load and spin a model

Game1.cs

Add ModelObject3D class and wire into Game1.



Game1.cs

C#

```
// ModelObject3D (add to Game1.cs or own file):
public class ModelObject3D:GameObject3D{
    Model _m; GraphicsDevice _gd;
    public ModelObject3D(Model m,GraphicsDevice gd){_m=m;_gd=gd;}
    public override void Update(float dt)
    { Rotation=Quaternion.Concatenate(Rotation,
        Quaternion.CreateFromAxisAngle(Vector3.Up,dt*0.5f));}
    public override void Draw(Camera3D cam,GraphicsDevice gd){
        Matrix w=World; float a=gd.Viewport.AspectRatio;
        foreach(ModelMesh mesh in _m.Meshes){
            foreach(BasicEffect fx in mesh.Effects){
                fx.EnableDefaultLighting();
                fx.World=w; fx.View=cam.GetView();
                fx.Projection=cam.GetProjection(a);}
            mesh.Draw();}}}

// In Game1 fields: Scene3D _scene=new();
// In LoadContent():
// Model ship=Content.Load<Model>("ship"); // add to MGCB first
// _scene.Add(new ModelObject3D(ship,GraphicsDevice));
// In Update(): _scene.Update(dt); _scene.Cleanup();
// In Draw() after Clear():
// GraphicsDevice.DepthStencilState=DepthStencilState.Default;
// _scene.Draw(_camera,GraphicsDevice);
```

■ Build & Run

\$ dotnet run

Expected output:

(3D model spins – fly around it with WASD+mouse)

■ Warning

DepthStencilState.Default MUST be set before 3D draws. SpriteBatch.End() resets it to None — 3D faces z-fight (flicker) without it.

➡ Exercises — Complete before moving on

➡ Exercises — Try These Now

- 1 Download 3 models from kenney.nl and place them at different positions.
- 2 Add a ground plane mesh at Y=-1.
- 3 Each model rotates at a different speed.
- 4 Implement Find<ModelObject3D>() and move it with arrow keys.
- 5 Add orbit path: update Position to circle origin at radius 3.

■ Quiz

Q1. Load a Model?

A. File.Read

✓ **Content.Load<Model>(name) after MGCB**

C. new Model()

D. SDL_Load

Answer: B

Q2. Iterate ModelMesh.Effects?

A. One effect only

✓ **Multiple meshes/materials possible**

C. Reset each frame

D. Required

Answer: B

Q3. Scene3D vs Scene2D?

A. Completely different

✓ **Identical loop — Draw takes Camera3D**

C. Scene3D faster

D. No buffer

Answer: B

Q4. DepthStencilState.Default?

A. Performance

✓ **SpriteBatch resets to None — 3D z-fights without it**

C. Required

D. Style

Answer: B

Q5. World = Scale*Rotate*Translate?

A. Random

✓ **SRT standard — other orders produce wrong results**

C. Required

D. Performance

Answer: B

PART 4

BUILDING GAMES

Apply every lesson to four complete projects

LESSON

27

Game 1 — Platformer: Player Controller

Coyote time, jump buffer, variable height

■ Concept

Coyote time: jump for 0.12s after leaving a ledge. **Jump buffer:** pressing Space 0.1s before landing fires on contact. **Variable height:** releasing Space early dampens upward velocity. Together these make platformer controls feel tight and fair.

Jump height (px) = $JUMP^2 / (2 \times GRAVITY)$. With GRAVITY=900, JUMP=430 → height≈103px≈3 tiles.

■ Project Setup — Create the Platformer project



Terminal

```
dotnet new console -n Platformer
cd Platformer
dotnet add package MonoGame.Framework.DesktopGL
```

■ CREATE FILE

PlatformerPlayer.cs

Player controller.

■ CREATE FILE

Game1.cs

Test with tile grid.

1

Create PlatformerPlayer.cs

PlatformerPlayer.cs

Create **PlatformerPlayer.cs**.



PlatformerPlayer.cs

C#

```
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class PlatformerPlayer{
    const float SPD=200f,JUMP=430f,GRAV=900f,FALL=700f,COY=0.12f,BUF=0.10f;
    public float X,Y,VX,VY;
    bool _grnd,_prevSp; float _coy,_buf;
    public PlatformerPlayer(float x,float y){X=x;Y=y;}
    public void Update(float dt,KeyboardState kb){
        VX=kb.IsKeyDown(Keys.Left)?-SPD:kb.IsKeyDown(Keys.Right)?SPD:0f;
        X+=VX*dt;
        if(!_grnd)VY=Math.Min(VY+GRAV*dt,FALL);
        Y+=VY*dt;
        _coy=_grnd?COY:Math.Max(0,_coy-dt);
        bool sp=kb.IsKeyDown(Keys.Space);
        if(sp&&!_prevSp)_buf=BUF; else _buf=Math.Max(0,_buf-dt);
        _prevSp=sp;
        if(_buf>0&&_coy>0){VY=-JUMP;_coy=0;_buf=0;}
        if(!sp&&VY<-150f)VY*=0.85f; // variable height
        _grnd=false;}
    public void SetGrounded(){_grnd=true;VY=0;}
    public Rectangle Bounds=>new((int)X,(int)Y,32,48);}
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Tip

_grnd=false at end of Update, reset by tile collision — handles walking off ledges automatically.

2

Test harness in Game1

Game1.cs

Create **Game1.cs** with hard-coded tiles.



```
using Microsoft.Xna.Framework;using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
public class Game1:Game{
    GraphicsDeviceManager _gfx;SpriteBatch _sb;
    Texture2D _px;PlatformerPlayer _p;
    const int TS=32;
    int[,] _map={{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
                {0,0,0,0,1,1,1,1,0,0,0,0,0,0,1,1,1,1,0,0,0,0},
                {0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,0,0,0,0,0,0,0},
                {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0},
                {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}};
    public Game1(){_gfx=new GraphicsDeviceManager(this);
        _gfx.PreferredBackBufferWidth=800;
        _gfx.PreferredBackBufferHeight=224;
        Content.RootDirectory="Content";}
    protected override void LoadContent(){
        _sb=new SpriteBatch(GraphicsDevice);
        _px=new Texture2D(GraphicsDevice,1,1);
        _px.SetData(new[]{Color.White});
        _p=new PlatformerPlayer(64,100);}
    protected override void Update(GameTime gt){
        float dt=Math.Min((float)gt.ElapsedGameTime.TotalSeconds,0.05f);
        _p.Update(dt,Keyboard.GetState());
        var pb=_p.Bounds;
        for(int ty=pb.Y/TS;ty<=(pb.Bottom-1)/TS;ty++){
            for(int tx=pb.X/TS;tx<=(pb.Right-1)/TS;tx++){
                if(ty<0||ty>=7||tx<0||tx>=25||_map[ty,tx]!=1)continue;
                var t=new Rectangle(tx*TS,ty*TS,TS,TS);
                if(!pb.Intersects(t))continue;
                float ox=Math.Min(pb.Right,t.Right)-Math.Max(pb.Left,t.Left);
                float oy=Math.Min(pb.Bottom,t.Bottom)-Math.Max(pb.Top,t.Top);
                if(oy<ox){if(pb.Center.Y<t.Center.Y){_p.Y-=oy;_p.SetGrounded();}
                    else _p.Y+=oy;}
                else _p.X+=pb.Center.X<t.Center.X?-ox:ox;
                pb=_p.Bounds;}
            base.Update(gt);}
    protected override void Draw(GameTime gt){
        GraphicsDevice.Clear(Color.SkyBlue);_sb.Begin();
        for(int ty=0;ty<7;ty++){for(int tx=0;tx<25;tx++){
            if(_map[ty,tx]==1)
                _sb.Draw(_px,new Rectangle(tx*TS,ty*TS,TS,TS),Color.SaddleBrown);
            _sb.Draw(_px,_p.Bounds,Color.DodgerBlue);
        }}
        _sb.End();base.Draw(gt);}}
```

■ Build & Run

\$ dotnet run

Expected output:

(Blue player, brown tiles. Arrows move, Space jumps.)

Test coyote: walk off ledge edge, press Space after – should jump.

Test buffer: press Space just before landing – fires on contact.

■ Tip

Test both mechanics deliberately. Coyote time and jump buffer are invisible to the player but make the difference between 'feels great' and 'feels unfair'.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Add dash: Shift launches 120px in movement direction, 1-second cooldown.
- 2 Add wall-jump: Space while touching a wall launches diagonally away.
- 3 Add double jump: second Space in air sets $VY = -JUMP * 0.7f$.
- 4 Add a coin (yellow 16×16) at (200,150). Overlap = score++.
- 5 Add moving platform: shifts left/right 3 tiles on a timer.

■ Quiz

Q1. Coyote time?

A. Infinite jumps

✓ **Jump briefly after leaving ledge**

C. Double jump

D. Wall jump

Answer: B

Q2. Jump buffer?

A. After landing

✓ **Just before landing — fires on contact**

C. Airborne only

D. 0.5s

Answer: B

Q3. Variable height: early release?

A. Stops player

✓ **Dampens upward velocity — shorter jump**

C. No effect

D. Kills all momentum

Answer: B

Q4. MAX_FALL prevents?

A. Jumping too high

✓ **Tunnelling through thin platforms**

C. Horizontal drift

D. Gravity

Answer: B

Q5. _grnd=false, reset by tile collision?

A. Required

✓ **Handles walking off ledges automatically**

C. Performance

D. Thread safety

Answer: B

LESSON

28

Game 1 — Platformer: Tiles and Camera

TileMap class and smooth camera follow

■ Concept

A **TileMap** wraps a 2D int array. Type 0=empty, 1=solid, 2=hazard. Only check tiles overlapping the player's bounding box (typically 4-6 tiles, not the full map). Camera follows with lerp and is clamped to map bounds.

■ Project Setup — Extend Platformer project

■ CREATE FILE

TileMap.cs

Tile map with culling.

■ CREATE FILE

Camera2D.cs

Copy from Lesson 11.

■ CREATE FILE

Game1.cs

Updated.

1

Create TileMap.cs

TileMap.cs

Create **TileMap.cs**.



TileMap.cs

C#

```
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Graphics;
public class TileMap{
    public int TS; int[,] _t;
    public int W=>_t.GetLength(1); public int H=>_t.GetLength(0);
    public TileMap(int[,] t,int ts=32){_t=t;TS=ts;}
    public int Get(int tx,int ty)
        =>(tx<0||ty<0||tx>=W||ty>=H)?0:_t[ty,tx];
    public bool IsSolid(int tx,int ty)=>Get(tx,ty)==1;
    public bool IsHazard(int tx,int ty)=>Get(tx,ty)==2;
    public Rectangle Bounds(int tx,int ty)=>new(tx*TS,ty*TS,TS,TS);
    public (int x0,int y0,int x1,int y1) Range(Rectangle r)=>
        (r.X/TS,r.Y/TS,(r.Right-1)/TS,(r.Bottom-1)/TS);
    public void Draw(SpriteBatch sb,Texture2D sol,Texture2D haz,
        Vector2 cam,int vpW,int vpH){
        int tx0=Math.Max(0,(int)(cam.X/TS));
        int ty0=Math.Max(0,(int)(cam.Y/TS));
        int tx1=Math.Min(W-1,tx0+vpW/TS+2);
        int ty1=Math.Min(H-1,ty0+vpH/TS+2);
        for(int ty=ty0;ty<=ty1;ty++)for(int tx=tx0;tx<=tx1;tx++){
            int t=Get(tx,ty);
            if(t==1)sb.Draw(sol,Bounds(tx,ty),Color.White);
            if(t==2)sb.Draw(haz,Bounds(tx,ty),Color.White);}}}
}
```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
    0 Warning(s)
```

■ Tip

Viewport culling is important for large maps. A 100×50 map has 5000 tiles — only ~600 are ever visible.

2

Add Camera2D and wire everything

Game1.cs

Update **Game1** to use **TileMap**, **Camera2D**, and hazard respawn.



Game1.cs

C#

```
// Fields: TileMap _map; Camera2D _cam;
// In LoadContent(), build larger map:
int[,] tiles=new int[14,60];
for(int tx=0;tx<60;tx++)tiles[13,tx]=1; // floor
for(int tx=3;tx<8;tx++) tiles[10,tx]=1; // platform
for(int tx=10;tx<14;tx++)tiles[13,tx]=2; // hazard
_map=new TileMap(tiles,32);
_cam=new Camera2D(_gfx);
_p=new PlatformerPlayer(64,300);
// In Update(), tile collision using TileMap.Range:
var pb=_p.Bounds;
var (tx0,ty0,tx1,ty1)=_map.Range(pb);
for(int ty=ty0;ty<=ty1;ty++)for(int tx=tx0;tx<=tx1;tx++){
    if(!_map.IsSolid(tx,ty))continue;
    var tile=_map.Bounds(tx,ty);
    if(!pb.Intersects(tile))continue;
    float ox=Math.Min(pb.Right,tile.Right)-Math.Max(pb.Left,tile.Left);
    float oy=Math.Min(pb.Bottom,tile.Bottom)-Math.Max(pb.Top,tile.Top);
    if(oy<ox){if(pb.Center.Y<tile.Center.Y){_p.Y-=oy;_p.SetGrounded();}
        else _p.Y+=oy;}
    else _p.X+=pb.Center.X<tile.Center.X?-ox:ox;
    pb=_p.Bounds;
    if(_map.IsHazard((int)(_p.X/32),(int)((_p.Y+47)/32)))
        {_p.X=64;_p.Y=300;} // hazard respawn
}
_cam.Follow(new Vector2(_p.X+16,_p.Y+24),7f,dt);
_cam.Clamp(_map.W*_map.TS,_map.H*_map.TS);
// In Draw(): _sb.Begin(transformMatrix:_cam.GetTransform());
// _map.Draw(_sb,_px,_px,_cam.Position,800,448);
// _sb.Draw(_px,_p.Bounds,Color.DodgerBlue);
// _sb.End();
```

■ Build & Run

\$ dotnet run

Expected output:

(Scrolling platformer. Camera follows. Hazard tiles respawn player.)

■ Tip

Follow target = player centre (X+16, Y+24 for 32×48 sprite). Using top-left offsets the camera.

⇒ Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add coins (list of positions). Overlap = removed, score++.
- 2 Add goal tile type 3. Reaching it prints 'Level Complete!'.
- 3 Show score with SpriteFont in top-left corner.
- 4 Add moving platform that shifts 3 tiles left/right.
- 5 Create a second level and switch on reaching the goal.

■ Quiz

Q1. Viewport culling?

A. Required

✓ Only renders visible tiles — skips off-screen

C. Sorts

D. Filters hazards

Answer: B

Q2. Camera lerp 7f?

A. Snaps

✓ 7× remaining distance/sec — smooth

C. 7px/frame

D. No lag

Answer: B

Q3. Camera clamp prevents?

A. Performance

✓ Showing empty space outside map

C. Hazard check

D. Input lag

Answer: B

Q4. Hazard check after collision resolution?

A. Always

B. Every frame

✓ After resolution — position is final

D. Before collision

Answer: C

Q5. Follow target at player centre vs top-left?

A. Same

✓ **Centre keeps player centred in view**

C. Required

D. Performance

Answer: B

LESSON

29

Game 2 — Shooter: Movement and Aim

8-direction movement and mouse-aimed shooting

■ Concept

Normalise 8-directional input before multiplying by speed — otherwise diagonal is 41% faster. Player rotates to face mouse cursor using **MathF.Atan2**. Bullets fire in the normalised direction from player to mouse world position.

■ Project Setup — Create the Shooter project



Terminal

```
dotnet new console -n Shooter
cd Shooter
dotnet add package MonoGame.Framework.DesktopGL
```

■ CREATE FILE

ShooterPlayer.cs

Movement, aim, shoot.

■ CREATE FILE

Game1.cs

Test harness.

1

Create ShooterPlayer.cs

ShooterPlayer.cs

Create **ShooterPlayer.cs**.



ShooterPlayer.cs

C#

```
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class ShooterPlayer{
    const float SPD=220f;
    public float X,Y,Angle;
    public ShooterPlayer(float x,float y){X=x;Y=y;}
    public Vector2 AimDir(Camera2D cam){
        var ms=Mouse.GetState();
        var wp=cam.ScreenToWorld(new Vector2(ms.X,ms.Y));
        var d=wp-new Vector2(X,Y);
        return d.LengthSquared(<0.01f?Vector2.UnitX:Vector2.Normalize(d));}
    public void Update(float dt,KeyboardState kb,Camera2D cam){
        float dx=0,dy=0;
        if(kb.IsKeyDown(Keys.W)||kb.IsKeyDown(Keys.Up))    dy=-1;
        if(kb.IsKeyDown(Keys.S)||kb.IsKeyDown(Keys.Down)) dy= 1;
        if(kb.IsKeyDown(Keys.A)||kb.IsKeyDown(Keys.Left)) dx=-1;
        if(kb.IsKeyDown(Keys.D)||kb.IsKeyDown(Keys.Right))dx= 1;
        float len=MathF.Sqrt(dx*dx+dy*dy);
        if(len>0){dx/=len;dy/=len;}
        X+=dx*SPD*dt; Y+=dy*SPD*dt;
        var aim=AimDir(cam);
        Angle=MathF.Atan2(aim.Y,aim.X);}
    public Rectangle Bounds=>new((int)X-16,(int)Y-16,32,32);}
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Warning

Normalise BEFORE multiplying by SPD. (1,1) unnormalised = $\text{length}\sqrt{2}\approx 1.41$ — diagonal 41% faster.

2

Wire into Game1

Game1.cs

Create **Game1.cs** with aim visualisation.



Game1.cs

C#

```
using Microsoft.Xna.Framework;using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
public class Game1:Game{
    GraphicsDeviceManager _gfx;SpriteBatch _sb;
    Texture2D _px;ShooterPlayer _p;Camera2D _cam;
    public Game1(){_gfx=new GraphicsDeviceManager(this);
        _gfx.PreferredBackBufferWidth=800;
        _gfx.PreferredBackBufferHeight=600;
        Content.RootDirectory="Content";IsMouseVisible=true;}
    protected override void LoadContent(){
        _sb=new SpriteBatch(GraphicsDevice);
        _px=new Texture2D(GraphicsDevice,1,1);
        _px.SetData(new[]{{Color.White}});
        _cam=new Camera2D(_gfx);_p=new ShooterPlayer(400,300);}
    protected override void Update(GameTime gt){
        if(Keyboard.GetState().IsKeyDown(Keys.Escape))Exit();
        float dt=Math.Min((float)gt.ElapsedGameTime.TotalSeconds,0.05f);
        _p.Update(dt,Keyboard.GetState(),_cam);
        _cam.Follow(new Vector2(_p.X,_p.Y),6f,dt);
        base.Update(gt);}
    protected override void Draw(GameTime gt){
        GraphicsDevice.Clear(Color.DarkSlateGray);
        _sb.Begin(transformMatrix:_cam.GetTransform());
        _sb.Draw(_px,_p.Bounds,Color.LimeGreen);
        var a=_p.AimDir(_cam);
        _sb.Draw(_px,new Rectangle((int)(_p.X+a.X*30),(int)(_p.Y+a.Y*30),4,4),Color.Yellow);
        _sb.End();base.Draw(gt);}}
```

■ Build & Run

\$ dotnet run

Expected output:

(Green square moves with WASD. Yellow dot always points at mouse cursor.)

■ Tip

Yellow dot at aim tip confirms ScreenToWorld is working correctly — it should track the cursor exactly.

⇒ Exercises — Complete before moving on

— Exercises — Try These Now

- 1 LMB click spawns a bullet moving in aim direction at 480px/s.
- 2 Screen-edge wrap: player exits one side, appears on opposite.
- 3 Dash: Shift launches 150px in movement direction, 0.8s cooldown.
- 4 Add 5 red enemy squares. Print 'Hit' when bullet touches one.
- 5 Draw a crosshair at mouse world position.

■ Quiz

Q1. Normalise movement vector?

A. Required

✓ Prevents diagonal 41% faster

C. Enables rotation

D. Required for bullets

Answer: B

Q2. Atan2(y,x)?

A. Distance

✓ Angle radians from X-axis to (x,y)

C. Magnitude

D. Dot product

Answer: B

Q3. ScreenToWorld needed?

A. Performance

✓ Raw mouse is screen space — must convert when camera is offset

C. Required

D. Style

Answer: B

Q4. Random pitch on shoot?

A. Fix

✓ Prevents robotic machine-gun sound

C. Required

D. Reduces lag

Answer: B

Q5. AimDir=Normalize(mouseWorld-playerPos)?

✓ **Yes — points from player to mouse**

- B. No — don't normalise
- C. Use screen coords
- D. Atan2 only

Answer: A

Game 2 — Shooter: Bullet Pool and Waves

Object pooling and wave spawning

■ Concept

A **bullet pool** pre-allocates a fixed array and reuses inactive bullets — no GC pauses during firing. A **wave manager** counts alive enemies, waits for all to die, then spawns the next wave with more enemies at higher speed.

■ Project Setup — Extend Shooter project

■ CREATE FILE **BulletPool.cs**

Fixed-size pool.

■ CREATE FILE **WaveManager.cs**

Wave spawning.

■ CREATE FILE **Game1.cs**

Connect.

1

Create BulletPool.cs and WaveManager.cs

BulletPool.cs

Create both files.

```

BulletPool.cs
C#
// BulletPool.cs
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Graphics;
public struct Bullet{
    public float X,Y,VX,VY,Life; public bool Active;
    public void Activate(float x,float y,float vx,float vy)
    {X=x;Y=y;VX=vx;VY=vy;Life=2f;Active=true;}
    public void Update(float dt)
    {if(!Active)return;X+=VX*dt;Y+=VY*dt;Life-=dt;if(Life<=0)Active=false;}}
public class BulletPool{
    const int SZ=128; Bullet[] _p=new Bullet[SZ];
    public void Fire(float x,float y,float vx,float vy)
    {foreach(ref var b in _p.AsSpan())if(!b.Active){b.Activate(x,y,vx,vy);return;}}
    public void Update(float dt){foreach(ref var b in _p.AsSpan())b.Update(dt);}
    public void Draw(SpriteBatch sb,Texture2D t)
    {foreach(var b in _p)if(b.Active)sb.Draw(t,
        new Rectangle((int)b.X-3,(int)b.Y-3,6,6),Color.Yellow);}
    public System.ReadOnlySpan<Bullet> All=>_p;}

// WaveManager.cs
using Microsoft.Xna.Framework; using System.Collections.Generic;
public class EnemyUnit{
    public float X,Y,Spd; public int HP=40; public bool Dead;
    public EnemyUnit(float x,float y,float s){X=x;Y=y;Spd=s;}
    public void Update(float dt,float px,float py){
        float dx=px-X,dy=py-Y,len=MathF.Sqrt(dx*dx+dy*dy);
        if(len>1f){X+=dx/len*Spd*dt;Y+=dy/len*Spd*dt;}}
    public Rectangle Bounds=>new((int)X-16,(int)Y-16,32,32);}
public class WaveManager{
    public int Wave{get;private set;}
    public List<EnemyUnit> Enemies=new();
    float _pause=3f; static System.Random _r=new();
    public void Update(float dt,float px,float py){
        Enemies.RemoveAll(e=>e.Dead);
        foreach(var e in Enemies)e.Update(dt,px,py);
        if(Enemies.Count==0){_pause-=dt;if(_pause<=0)Spawn();}}
    void Spawn(){
        Wave++;_pause=3f;
        for(int i=0;i<3+Wave*2;i++){
            float ex=_r.NextSingle(>0.5f?-30:830);
            float ey=(float)(_r.NextDouble()*600);
            Enemies.Add(new EnemyUnit(ex,ey,55f+Wave*12f));}}}

```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.

0 Warning(s)

■ Tip

`foreach(ref var b in _p.AsSpan())` — `ref` modifies struct in-place. Without `ref`, `b` is a copy and changes are discarded.

2

Connect in Game1

Game1.cs

Add fields and wire bullets, waves, collision.

```
Game1.cs C#
BulletPool _bp=new(); WaveManager _wm=new();
int _score=0; bool _prevLMB;
// In Update():
bool lmb=Mouse.GetState().LeftButton==ButtonState.Pressed;
if(lmb&& !_prevLMB){var d=_p.AimDir(_cam);
    _bp.Fire(_p.X,_p.Y,d.X*480f,d.Y*480f);}
_prevLMB=lmb;
_bp.Update(dt);
_wm.Update(dt,_p.X,_p.Y);
foreach(var e in _wm.Enemies)
    foreach(ref var b in _bp.All)
        if(b.Active&&e.Bounds.Contains((int)b.X,(int)b.Y))
            {e.HP-=20;b.Active=false;
                if(e.HP<=0){e.Dead=true;_score+=100;}}
Window.Title=$"Wave:{_wm.Wave} Score:{_score}";
// In Draw(): _bp.Draw(_sb,_px);
// foreach(var e in _wm.Enemies)_sb.Draw(_px,e.Bounds,Color.Red);
```

■ Build & Run

```
$ dotnet run
```

Expected output:

```
(Enemies chase player. Click shoots. Enemies die. Wave counter advances.)
```

■ Tip

`Bounds.Contains(bulletPos)` = point-in-rectangle. Faster than full AABB for small fast bullets.

⇒ Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Player health: touching enemy deals 10 dmg. 0 HP = 'Game Over'.
- 2 Score multiplier that increases with each wave.
- 3 Nuke pickup (green circle): collecting destroys all enemies.
- 4 Spread shot: Shift fires 3 bullets 20° apart.
- 5 Display bullet count remaining in window title.

■ Quiz

Q1. Pre-allocate pool?

A. Required

✓ Avoids GC pauses — no heap allocation during firing

C. Faster code

D. Required

Answer: B

Q2. ref in foreach(ref var b)?

A. Style

✓ Modifies struct in-place — without ref changes are lost

C. Performance

D. Required for span

Answer: B

Q3. WaveManager spawns when?

A. Every 30s

✓ All enemies dead AND pause elapsed

C. Fixed count

D. Score

Answer: B

Q4. Contains vs Intersects?

A. Same

✓ Contains=point-in-rect faster for small bullets. Intersects=rect-vs-rect

C. Contains less accurate

D. Intersects always better

Answer: B

Q5. Wave speed increases?

A. Bug

✓ **Difficulty scaling — each wave faster**

C. Random

D. Required

Answer: B

LESSON

31

Game 3 — FPS: Camera and Movement

First-person camera and gravity

■ Concept

Lock mouse with `Mouse.SetPosition` each frame. Delta = current – centre → yaw/pitch. WASD uses flat forward (Y=0). Gravity applied manually; ground at Y=1.8 (eye height).

■ Project Setup — Create FPSGame project



Terminal

```
dotnet new console -n FPSGame
cd FPSGame
dotnet add package MonoGame.Framework.DesktopGL
```

■ CREATE FILE

FPSController.cs

First-person controller.

■ CREATE FILE

Game1.cs

3D room.

1

Create FPSController.cs

FPSController.cs

Create **FPSController.cs**. (Camera3D from Lesson 21 must be in the project.)



FPSController.cs

C#

```
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class FPSController{
    Camera3D _cam; float _vy=0f; bool _grnd=false;
    int _cx,_cy; const float SPD=4f,JUMP=5f;
    public Camera3D Camera=>_cam;
    public FPSController(int w,int h){
        _cam=new Camera3D{Position=new(0,1.8f,0)};
        _cx=w/2;_cy=h/2;Mouse.SetPosition(_cx,_cy);}
    public void Update(float dt,KeyboardState kb){
        var ms=Mouse.GetState();
        _cam.ProcessMouseDelta(ms.X-_cx,ms.Y-_cy);
        Mouse.SetPosition(_cx,_cy);
        _cam.MoveWASD(kb,SPD,dt);
        if(!_grnd)_vy-=9.8f*dt;
        var p=_cam.Position; p.Y+=_vy*dt; _cam.Position=p;
        if(_cam.Position.Y<1.8f){
            p=_cam.Position;p.Y=1.8f;_cam.Position=p;
            _grnd=true;_vy=0f;}
        else _grnd=false;
        if(kb.IsKeyDown(Keys.Space)&&_grnd){_vy=JUMP;_grnd=false;}}
```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Tip

Camera Y=1.8 = eye height. Tune to match your world scale.

2

Simple room in Game1

Game1.cs

Create **Game1.cs** with a floor quad.

```
Game1.cs C#
using Microsoft.Xna.Framework;using Microsoft.Xna.Framework.Graphics;
using Microsoft.Xna.Framework.Input;
public class Game1:Game{
    GraphicsDeviceManager _gfx; BasicEffect _fx;
    FPSController _ctrl; VertexBuffer _vb; IndexBuffer _ib;
    public Game1(){_gfx=new GraphicsDeviceManager(this);
        _gfx.PreferredBackBufferWidth=800;
        _gfx.PreferredBackBufferHeight=600;
        Content.RootDirectory="Content";}
    protected override void LoadContent(){
        _ctrl=new FPSController(800,600);
        _fx=new BasicEffect(GraphicsDevice);
        _fx.EnableDefaultLighting();
        _fx.DiffuseColor=Color.SlateGray.ToVector3();
        var v=new VertexPositionNormalTexture[4];
        v[0]=new(new Vector3(-10,0,-10),Vector3.Up,new Vector2(0,0));
        v[1]=new(new Vector3( 10,0,-10),Vector3.Up,new Vector2(1,0));
        v[2]=new(new Vector3( 10,0, 10),Vector3.Up,new Vector2(1,1));
        v[3]=new(new Vector3(-10,0, 10),Vector3.Up,new Vector2(0,1));
        _vb=new VertexBuffer(GraphicsDevice,
            VertexPositionNormalTexture.VertexDeclaration,4,BufferUsage.WriteOnly);
        _vb.SetData(v);
        short[] idx={0,2,1,0,3,2};
        _ib=new IndexBuffer(GraphicsDevice,IndexElementSize.SixteenBits,6,BufferUsage.WriteOnly);
        _ib.SetData(idx);}
    protected override void Update(GameTime gt){
        if(Keyboard.GetState().IsKeyDown(Keys.Escape))Exit();
        float dt=Math.Min((float)gt.ElapsedGameTime.TotalSeconds,0.05f);
        _ctrl.Update(dt,Keyboard.GetState()); base.Update(gt);}
    protected override void Draw(GameTime gt){
        GraphicsDevice.Clear(Color.Black);
        GraphicsDevice.DepthStencilState=DepthStencilState.Default;
        _fx.View=_ctrl.Camera.GetView();
        _fx.Projection=_ctrl.Camera.GetProjection(
            GraphicsDevice.Viewport.AspectRatio);
        _fx.World=Matrix.Identity;
        GraphicsDevice.SetVertexBuffer(_vb);
        GraphicsDevice.Indices=_ib;
        foreach(var p in _fx.CurrentTechnique.Passes){
            p.Apply();
            GraphicsDevice.DrawIndexedPrimitives(PrimitiveType.TriangleList,0,0,2);}
        base.Draw(gt);}}
```

■ Build & Run

\$ dotnet run

Expected output:

(Gray floor. WASD moves. Mouse looks. Space jumps. Escape closes.)

■ Tip

Add wall quads by adding more VertexPositionNormalTexture arrays with appropriate normals.

⇒ Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Add 4 wall quads and a ceiling quad to complete the room.
- 2 Add a box mesh at (0,0.5,-3) — walk up to it.
- 3 Head bob: oscillate Y by $\text{MathF.Sin}(t*10)*0.05f$ while moving.
- 4 Sprint: Shift doubles speed, reduces FOV to 75 with Lerp.
- 5 Press E near a door (within 1.5 units) to 'open' it.

■ Quiz

Q1. Camera Y=1.8?

A. Bug

✓ Eye height — camera represents eyes not feet

C. Required

D. Performance

Answer: B

Q2. Flat forward Y=0?

A. Slow

✓ Prevents flying when looking up/down

C. Required for gravity

D. Style

Answer: B

Q3. Re-centre mouse prevents?

A. Lag

✓ Cursor hitting screen edge and stopping rotation

C. Leak

D. Required

Answer: B

Q4. DepthStencilState.Default?

A. Performance

✓ **SpriteBatch resets to None — z-fight without it**

C. Required

D. Style

Answer: B

Q5. _grnd=false, set by floor check?

A. Required

✓ **Handles stepping off ledges without special case**

C. Performance

D. Thread

Answer: B

Game 3 — FPS: Raycasting and Interaction

Ray-AABB for looking at objects

■ Concept

Cast a ray from camera centre each frame. Test against interactables. Return closest hit within range. E key triggers it. The **slab method**: per-axis entry (tNear) and exit (tFar). If $\max(tNear) \leq \min(tFar)$ and $tFar > 0$, the ray hit the box.

■ Project Setup — Extend FPSGame

■ CREATE FILE

Raycaster.cs

Ray-AABB test.

■ CREATE FILE

Interactable.cs

Interface + Door.

■ CREATE FILE

Game1.cs

Cast ray each frame.

1

Create Raycaster.cs and Interactable.cs

Raycaster.cs

Create both files.

```

Raycaster.cs
// Raycaster.cs
using Microsoft.Xna.Framework;
public static class Raycaster{
    public struct Hit{public bool IsHit;public float T;}
    public static Hit Test(Vector3 ro,Vector3 rd,Vector3 mn,Vector3 mx){
        Vector3 inv=new(
            rd.X!=0?1f/rd.X:float.MaxValue,
            rd.Y!=0?1f/rd.Y:float.MaxValue,
            rd.Z!=0?1f/rd.Z:float.MaxValue);
        Vector3 t0=(mn-ro)*inv; Vector3 t1=(mx-ro)*inv;
        Vector3 tN=Vector3.Min(t0,t1); Vector3 tF=Vector3.Max(t0,t1);
        float tn=Math.Max(Math.Max(tN.X,tN.Y),tN.Z);
        float tf=Math.Min(Math.Min(tF.X,tF.Y),tF.Z);
        return new Hit{IsHit=tn<=tf&&tf>0,T=tn};}}

// Interactable.cs
using Microsoft.Xna.Framework;
public interface IInteractable{
    Vector3 BoxMin{get;} Vector3 BoxMax{get;}
    string Prompt{get;} void Interact();}
public class Door:IInteractable{
    public Vector3 Pos; bool _open;
    public Door(Vector3 p){Pos=p;}
    public Vector3 BoxMin=>Pos+new Vector3(-0.3f,0,-0.1f);
    public Vector3 BoxMax=>Pos+new Vector3(0.3f,2,0.1f);
    public string Prompt=>_open?"[E] Close":"[E] Open";
    public void Interact()
    {_open=!_open;System.Console.WriteLine(_open?"Opened":"Closed");}}

```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Tip

tNear=entry distance, tFar=exit distance. $tNear \leq tFar$ = ray passed through box.

2

Cast ray in Game1

Game1.cs

Add fields and Update code.



Game1.cs

C#

```
System.Collections.Generic.List<IInteractable> _objs=new();
IInteractable _looked; bool _prevE;
// In LoadContent(): _objs.Add(new Door(new Vector3(0,0,-4)));
// In Update(), after controller:
_looked=null; float best=3f;
var ro=_ctrl.Camera.Position;
var rd=_ctrl.Camera.Forward;
foreach(var o in _objs){
    var h=Raycaster.Test(ro,rd,o.BoxMin,o.BoxMax);
    if(h.IsHit&&h.T<best){best=h.T;_looked=o;}}
bool e=Keyboard.GetState().IsKeyDown(Keys.E);
if(e&&(!_prevE&&_looked!=null)_looked.Interact());
_prevE=e;
Window.Title=_looked!=null?_looked.Prompt:"FPS Demo";
```

■ Build & Run

\$ dotnet run

Expected output:

(Look at door – title shows '[E] Open'. Press E – prints Opened.)

■ Tip

Range 3 units. To vary per object, add a Range property to IInteractable.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 Add Chest that prints its contents on interact.
- 2 Add lever that toggles bool and changes light direction.
- 3 Crosshair: draw small cross in screen centre when looking at interactable.
- 4 RaycastAll: return all hits sorted by distance.
- 5 Explosive barrel: 5s countdown on interact, then 'BOOM' and remove.

■ Quiz

Q1. Slab method finds?

A. Normal

✓ Entry and exit distances through AABB

C. Closest vertex

D. UV

Answer: B

Q2. tNear > tFar means?

A. Hit

✓ Missed the box

C. Inside box

D. Parallel

Answer: B

Q3. Return closest hit?

A. Performance

✓ Prevents triggering objects behind closer wall

C. Required

D. Simpler

Answer: B

Q4. Interactable decouples?

A. Physics/rendering

✓ Raycaster from specific types — any class can be interactable

C. Input/movement

D. Camera/scene

Answer: B

Q5. hit.T < 3f?

A. 3 frames

✓ Ignore hits farther than 3 world units

C. 3 second timeout

D. 3 pixels

Answer: B

LESSON

33

Game 4 — TPS: Orbit Camera and Movement

Spherical orbit camera and camera-relative movement

■ Concept

Orbit camera: position from spherical coordinates (yaw, pitch, distance). Character moves in camera's horizontal facing direction. **Quaternion.Slerp** smoothly rotates the character to face movement direction each frame.

■ Project Setup — Create TPSTGame project



Terminal

```
dotnet new console -n TPSTGame
cd TPSTGame
dotnet add package MonoGame.Framework.DesktopGL
```

■ CREATE FILE

OrbitCamera.cs

Spherical orbit camera.

■ CREATE FILE

TPSTPlayer.cs

Camera-relative movement.

■ CREATE FILE

Game1.cs

3D game loop.

1

Create OrbitCamera.cs

OrbitCamera.cs

Create **OrbitCamera.cs**.

```

OrbitCamera.cs
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class OrbitCamera{
    public Vector3 Target=Vector3.Zero;
    public float Dist=5f,Yaw=0f,Pitch=0.4f;
    int _cx,_cy; const float S=0.004f;
    public void Init(int w,int h){_cx=w/2;_cy=h/2;Mouse.SetPosition(_cx,_cy);}
    public void Update(float dt){
        var ms=Mouse.GetState();
        Yaw+=(ms.X-_cx)*S;
        Pitch=MathHelper.Clamp(Pitch-(ms.Y-_cy)*S,0.1f,1.2f);
        Mouse.SetPosition(_cx,_cy);}
    public Vector3 Position=>Target+new Vector3(
        Dist*MathF.Cos(Pitch)*MathF.Sin(Yaw),
        Dist*MathF.Sin(Pitch),
        Dist*MathF.Cos(Pitch)*MathF.Cos(Yaw));
    public Matrix GetView()=>Matrix.CreateLookAt(Position,Target,Vector3.Up);
    public Matrix GetProjection(float a)=>
        Matrix.CreatePerspectiveFieldOfView(MathHelper.ToRadians(60f),a,0.1f,1000f);
    public Vector3 ForwardFlat=>Vector3.Normalize(new(-MathF.Sin(Yaw),0,-MathF.Cos(Yaw)));
    public Vector3 RightFlat=>Vector3.Normalize(Vector3.Cross(ForwardFlat,Vector3.Up));}

```

■ Build & Run

\$ dotnet build

Expected output:

Build succeeded.
0 Warning(s)

■ Tip

Pitch clamped 0.1–1.2 rad ($\approx 6^\circ$ – 69°) prevents flipping upside down or going below ground.

2

Create TPSPlayer and wire Game1

TPSPlayer.cs

Create **TPSPlayer.cs** and update Game1.

```

TPSPlayer.cs
// TPSPlayer.cs
using Microsoft.Xna.Framework; using Microsoft.Xna.Framework.Input;
public class TPSPlayer{
    public Vector3 Position=Vector3.Zero;
    public Quaternion Facing=Quaternion.Identity;
    float _vy=0f; const float SPD=4f,JUMP=5f;
    public void Update(float dt,KeyboardState kb,OrbitCamera cam){
        Vector3 mv=Vector3.Zero;
        if(kb.IsKeyDown(Keys.W))mv+=cam.ForwardFlat;
        if(kb.IsKeyDown(Keys.S))mv-=cam.ForwardFlat;
        if(kb.IsKeyDown(Keys.A))mv-=cam.RightFlat;
        if(kb.IsKeyDown(Keys.D))mv+=cam.RightFlat;
        if(mv.LengthSquared(>)>0.01f){
            mv=Vector3.Normalize(mv);
            Position+=mv*SPD*dt;
            var tr=Quaternion.CreateFromRotationMatrix(
                Matrix.CreateWorld(Vector3.Zero,mv,Vector3.Up));
            Facing=Quaternion.Slerp(Facing,tr,10f*dt);}
        _vy-=9.8f*dt; Position.Y+=_vy*dt;
        if(Position.Y<0f){Position.Y=0f;_vy=0f;}
        if(kb.IsKeyDown(Keys.Space)&&Position.Y<=0.01f)_vy=JUMP;}
    public Matrix World=>Matrix.CreateFromQuaternion(Facing)
        *Matrix.CreateTranslation(Position);}

// Game1 additions:
// Fields: OrbitCamera _cam=new(); TPSPlayer _player=new();
// LoadContent: _cam.Init(800,600);
// Update: _cam.Update(dt); _player.Update(dt,kb,_cam);
//         _cam.Target=_player.Position+Vector3.Up;
// Draw: _fx.View=_cam.GetView();
//       _fx.Projection=_cam.GetProjection(aspect);
//       _fx.World=_player.World; _mesh.Draw(_fx);

```

■ Build & Run

\$ dotnet run

Expected output:

(WASD moves player relative to camera. Camera orbits with mouse. Player turns smoothly.)

■ Tip

_cam.Target = _player.Position + Vector3.Up aims at torso, not feet. Without this the camera looks at the ground.

— Exercises — Complete before moving on

⇒ Exercises — Try These Now

- 1 Sprint: Shift × 2.5 speed, FOV increases to 75.
- 2 Spring arm: raycast player→camera, pull forward if blocked.
- 3 Lock-on: Tab finds nearest enemy within 10 units.
- 4 Dodge roll: double-tap direction to roll 2 units quickly.
- 5 Print Idle/Walk/Run state based on movement speed each frame.

■ Quiz

Q1. Spherical coordinates define?

A. Rotation matrix

✓ Camera on sphere around target

C. Quaternion

D. Frustum

Answer: B

Q2. ForwardFlat Y=0?

A. Performance

✓ Keeps movement horizontal

C. Required

D. Style

Answer: B

Q3. Slerp(Facing,target,10*dt)?

A. Snaps

✓ Smooth 10× remaining angle/sec

C. Angular velocity

D. Mirrors

Answer: B

Q4. _cam.Target = pos + Up?

A. Required

✓ Aims at torso not feet — looks natural

C. Performance

D. CreateLookAt

Answer: B

Q5. Spring arm prevents?

A. Flying

✓ **Camera clipping into geometry**

C. Gimbal lock

D. Animation

Answer: B

LESSON

34

Game 4 — TPS: Melee Combat and Enemy AI

Attack hitbox, knockback, three-state AI

■ Concept

Melee uses a temporary sphere hitbox (0.25s active). Knockback direction = $\text{normalise}(\text{enemy} - \text{attacker}) \times \text{force}$. Enemy AI: Patrol→Chase→Attack driven by distance-squared thresholds. `_atkTimer` enforces a 1.5s cooldown.

■ Project Setup — Extend TPSTGame

■ CREATE FILE

AttackHitbox.cs

Sphere hitbox.

■ CREATE FILE

MeleeEnemy.cs

Three-state AI.

■ CREATE FILE

Game1.cs

Combat loop.

1

Create AttackHitbox.cs and MeleeEnemy.cs

AttackHitbox.cs

Create both files.

```

AttackHitbox.cs
// AttackHitbox.cs
using Microsoft.Xna.Framework;
public class AttackHitbox{
    public Vector3 Centre; public float Radius,Life=0f;
    public bool Active=>Life>0f;
    public void Activate(Vector3 c,float r,float dur){Centre=c;Radius=r;Life=dur;}
    public void Update(float dt)=>Life=Math.Max(0,Life-dt);
    public bool Hits(Vector3 p,float r)
    {if(!Active)return false;float s=Radius+r;
    return Vector3.DistanceSquared(Centre,p)<=s*s;}}

// MeleeEnemy.cs
using Microsoft.Xna.Framework;
public class MeleeEnemy{
    enum S{Patrol,Chase,Attack}
    S _s=S.Patrol; Vector3 _pat; float _at=0f;
    static System.Random _r=new();
    public Vector3 Pos,Vel=Vector3.Zero;
    public int HP=80; public bool Dead;
    public MeleeEnemy(Vector3 p){Pos=p;_pat=p;}
    public void Update(float dt,Vector3 pp){
        float d2=Vector3.DistanceSquared(Pos,pp);
        _s=d2<2.25f?S.Attack:d2<64f?S.Chase:S.Patrol;
        switch(_s){
        case S.Patrol:
            Move(_pat,1.5f,dt);
            if(Vector3.DistanceSquared(Pos,_pat)<0.25f)
                _pat=new((float)(_r.NextDouble()*10-5),0,(float)(_r.NextDouble()*10-5));
            break;
        case S.Chase: Move(pp,3f,dt); break;
        case S.Attack:
            _at-=dt;
            if(_at<=0f){System.Console.WriteLine("Enemy attacks!");_at=1.5f;}
            break;}}
    void Move(Vector3 t,float spd,float dt)
    {var d=Vector3.Normalize(t-Pos);Pos+=d*spd*dt;}
    public void TakeDmg(int dmg,Vector3 from)
    {HP-=dmg;Vel=Vector3.Normalize(Pos-from)*4f;if(HP<=0)Dead=true;}}

```

■ Build & Run

```
$ dotnet build
```

Expected output:

```
Build succeeded.
    0 Warning(s)
```

■ Tip

Distance thresholds use DistanceSquared ($1.5^2=2.25$, $8^2=64$) — no sqrt needed.

Add fields and combat loop.

```

Game1.cs
AttackHitbox _hb=new();
System.Collections.Generic.List<MeleeEnemy> _enemies=new();
int _score=0; bool _prevJ;
// In LoadContent(), spawn 4 enemies:
var r=new System.Random(7);
for(int i=0;i<4;i++)
    _enemies.Add(new MeleeEnemy(new(
        (float)(r.NextDouble()*12-6),0,
        (float)(r.NextDouble()*12-6))));
// In Update():
_hb.Update(dt);
_enemies.RemoveAll(e=>e.Dead);
foreach(var e in _enemies){
    e.Update(dt,_player.Position);
    e.Pos+=e.Vel*dt;
    e.Vel=Vector3.Lerp(e.Vel,Vector3.Zero,5f*dt);}
bool j=Keyboard.GetState().IsKeyDown(Keys.J);
if(j&& !_prevJ){
    // Spawn hitbox 1.2 units in front of player
    var fwd=new Vector3(
        2*_player.Facing.Y,0,-2*_player.Facing.W);
    if(fwd.LengthSquared()<0.01f)fwd=Vector3.Forward;
    fwd=Vector3.Normalize(fwd);
    _hb.Activate(_player.Position+fwd*1.2f+Vector3.Up,1f,0.25f);}
_prevJ=j;
foreach(var e in _enemies)
    if(_hb.Hits(e.Pos+Vector3.Up*0.5f,0.5f))
        {e.TakeDmg(25,_player.Position);_score+=50;}
Window.Title=$"Score:{_score} Enemies:{_enemies.Count}";
// In Draw(): draw enemy boxes

```

■ Build & Run

```
$ dotnet run
```

Expected output:

(Enemies patrol/chase/attack. Press J to swing – enemies take damage and are knocked back.)

■ Tip

`e.Vel=Vector3.Lerp(e.Vel,Vector3.Zero,5f*dt)` applies drag — knockback decays over ~0.2s.

— Exercises — Complete before moving on

— Exercises — Try These Now

- 1 HP bar above each enemy: billboard quad scaled by HP/maxHP.
- 2 Charge attack: hold J for 0.5s → larger hitbox (radius 2, dmg 60).
- 3 Enemies attack back: in Attack state, create their own hitbox.
- 4 Dodge: double-tap for 2-unit roll, invincible 0.3s.
- 5 Score display with SpriteFont in top-left.

■ Quiz

Q1. Hitbox duration 0.25s?

- A. 1 second
- B. 5 seconds
- ✓ C. 0.2-0.3s — animation window
- D. Until next attack

Answer: C

Q2. DistanceSquared thresholds?

- A. Required
- ✓ B. Avoids sqrt — same result
- C. Accurate
- D. Negatives

Answer: B

Q3. Knockback=normalize(e-a)*force?

- A. Required
- ✓ B. Away from attacker, consistent force
- C. Attract
- D. Gravity

Answer: B

Q4. Lerp(vel,zero,5*dt)?

- A. Jump
- ✓ B. Drag — decelerates knockback over time
- C. Acceleration
- D. Random

Answer: B

Q5. `_atkTimer` prevents?

- A. Patrol
- ✓ **Every-frame attack — enforces cooldown**
- C. Chasing
- D. Hitbox

Answer: B